UAV-NOMA Disaster Network Planning Simulator

A simulation-based planning tool for UAV base stations using NOMA in disaster areas, with a fair evaluation of reinforcement-learning control

A major project report submitted in partial fulfilment of the requirements for the award of the degree of

Bachelor of Engineering / Bachelor of Technology in [PENDING: branch, e.g. Electronics and Communication Engineering]

Submitted by

[PENDING: Student name 1, USN / roll number]

[PENDING: Student name 2, USN / roll number]

[PENDING: Student name 3, USN / roll number]

[PENDING: Student name 4, USN / roll number]

Under the guidance of

[PENDING: Guide name, designation, department]

[PENDING: Department of ..., College name, City]

[PENDING: Affiliated university]

[PENDING: Academic year, e.g. 2026-2027]

Certificate

[PENDING: the university's certificate text, signed by the guide, the head of department and the principal, and the external examiner's signature block, in the format prescribed by the university.]

This is to certify that the project work entitled "UAV-NOMA Disaster Network Planning Simulator: a simulation-based planning tool for UAV base stations using NOMA in disaster areas" is a bonafide work carried out by [PENDING: names and USNs] in partial fulfilment of the requirements for the award of the degree of [PENDING: degree] during the academic year [PENDING: year].

Guide	Head of the Department	Principal
[PENDING: name, signature]	[PENDING: name, signature]	[PENDING: name, signature]

Declaration

We, the students named on the title page, declare that the work presented in this report is our own, carried out under the guidance of [PENDING: guide], and has not been submitted for any other degree. Sources are cited in the References.

Disclosure of AI assistance [PENDING: confirm the wording against the university's policy]: much of the simulator, the baseline pilots and the reinforcement-learning pipeline was first written in AI-assisted sessions (an AI coding assistant) with the project owner, and the team then took ownership to review, explain and extend the code; every result was re-run, checked and recorded in the project book.

[PENDING: place, date, signatures]

Acknowledgement

[PENDING: thanks to the guide, the head of department, the principal, the institution, family and friends.] The idea of a moving base station using NOMA came from our guide; the team extended it with moving crowds, collapsed buildings, a battery limit and a learning controller.

Abstract

When an earthquake or a flood destroys the mobile towers in an area, a drone carrying a small base station can restore service for a while, but where it should fly, how high, and how it should share its radio resources are open questions. We built an open, tested, simulation-based planning tool for this problem: moving survivor crowds with five service classes, collapsed buildings, the Al-Hourani air-to-ground channel, the Zeng-Xu-Zhang rotary-wing energy model with a battery, and a NOMA downlink in which two users share one frequency slice. Every strategy flies the same seeded scenarios and is compared with paired 95% confidence intervals. Results: a drone that follows the crowd serves 74 of every 100 people where a parked drone serves 43; when NOMA and OMA are optimised in the same way, adaptive NOMA serves 3 to 5 more people per 100 for every pilot; circling at the minimum-power speed serves about 26% more people per battery than hovering. A reinforcement-learning pilot that sees only what a real drone could measure did not reach its first gate (0.576 against a pass mark of 0.580). A "clairvoyant" pilot that knows the crowd's future showed why: in walking-crowd scenarios there is almost no room for anticipation (+0.002). In redesigned scenarios with demand switching between sites 2 km apart there is room, but the first round of learning agents still scored 0.073 to 0.137 below the best simple pilots, so the next step is a hybrid in which the AI only decides when to leave. A simulated quadcopter with an autopilot flew the same plans within 0.012 coverage of the simplified game. We report where learning helps, where it does not, and what remains open.

Keywords: UAV base station, NOMA, disaster communication, air-to-ground channel, rotary-wing energy, reinforcement learning, baselines, simulation-based planning.

Contents

Certificate 2

Declaration 3

Acknowledgement 4

Abstract 5

List of figures 8

List of tables 8

Chapter 1. Introduction 9

1.1 Problem statement . . . 9

1.2 Motivation . . . 9

1.3 Objectives . . . 9

1.4 Hypotheses . . . 10

1.5 Scope . . . 10

1.6 Organisation of the report . . . 10

Chapter 2. Literature survey 11

2.1 UAV base-station placement . . . 11

2.2 The air-to-ground channel . . . 11

2.3 Rotary-wing energy . . . 11

2.4 NOMA and UAV-NOMA . . . 11

2.5 Reinforcement learning for UAV networks . . . 11

2.6 Evaluation practice . . . 12

2.7 Literature matrix . . . 12

2.8 Gap and positioning . . . 13

Chapter 3. System design 14

3.1 Architecture . . . 14

3.2 Scenarios . . . 14

3.3 The radio channel . . . 15

3.4 NOMA and OMA . . . 15

3.5 Energy and battery . . . 16

3.6 The game (Gymnasium environment) . . . 16

3.7 Pilots . . . 16

3.8 The AI . . . 17

3.9 The evaluation protocol . . . 17

3.10 The dashboard and the 3D quadcopter . . . 18

Chapter 4. Implementation 19

4.1 Tools . . . 19

4.2 Code structure . . . 19

4.3 Testing and continuous integration . . . 19

4.4 Reproducibility . . . 19

Chapter 5. Results **21**

5.1 Physics checks (Gate G2) . . . 21

5.2 H1: an adaptive drone beats a parked one . . . 21

5.3 H3: NOMA against OMA . . . 23

5.4 Energy: circling beats hovering on a tight battery . . . 25

5.5 Why the AI stalls on a still crowd: the information ladder . . . 26

5.6 RL on a still crowd: Gate G4 (negative result) . . . 26

5.7 Is there room for an anticipating AI? Clairvoyant headroom and competing sites . . . 29

5.8 RL on competing sites: Gate G5, round 1 (negative result) . . . 30

5.9 Sim-to-sim: does the simplified flight model change the results? . . . 31

5.10 Value of location information . . . 32

5.11 Real-place scenario . . . 32

Chapter 6. Discussion **33**

6.1 Where learning helps and where it does not . . . 33

6.2 H2 so far . . . 33

6.3 Limitations . . . 33

6.4 Threats to validity . . . 34

Chapter 7. Conclusion and future work **35**

7.1 Conclusion . . . 35

7.2 Future work . . . 35

References **37**

List of figures

No.	Title	Section
3.1	System architecture: one physics engine, one game, many pilots	3.1
3.2	One second of the game	3.3
3.3	How each pilot reaches the game	3.7
3.4	The evaluation protocol and the gates	3.9
3.5	The 3D quadcopter in the loop (sim-to-sim)	3.10
5.1	Four training scenarios: crowds, scattered users and rescue users	5.1
5.2	Best altitude per user class	5.1
5.3	Seven pilots on the `hard` profile	5.2
5.4	Pilots on one hard scenario with moving crowds	5.2
5.5	NOMA vs OMA in the game, hard profile	5.3
5.6	NOMA pairs on the map and every user's rate	5.3
5.7	Four access schemes, hard profile	5.3
5.8	Four access schemes, low-power profile	5.3
5.9	First AI pilot (velocity control) drifting away from the crowd	5.6
5.10	Waypoint control with a 10-second decision rhythm	5.6
5.11	Best AI (E0) vs grid search on validation scenario 105	5.6
5.12	Circling vs hovering	5.4
5.13	Room for a pilot that thinks ahead, fresh scenarios	5.7
5.14	The 3D flight page (simulated quadcopter)	5.9

List of tables

No.	Title	Section
1.1	Hypotheses	1.4
2.1	Literature matrix	2.7
3.1	Scenario profiles	3.2
3.2	User classes and minimum rates	3.2
3.3	The pilots	3.7
4.1	Code structure	4.2
4.2	Dataset tables	4.4
5.1	H1: pilots on the `hard` profile	5.2
5.2	H3: fixed NOMA vs OMA in the game	5.3
5.3	H3: adaptive NOMA vs equally optimised OMA	5.3
5.4	Circling vs hovering	5.4
5.5	The information ladder	5.5
5.6	Gate G4 runs	5.6
5.7	Clairvoyant headroom on walking crowds	5.7
5.8	Room in the competing-sites designs	5.7
5.9	Room confirmed on fresh scenarios	5.7
5.10	G5 round 1	5.8
5.11	Sim-to-sim check	5.9
6.1	Where each tool fits	6.1
7.1	Hypotheses: verdicts	7.1

Chapter 1. Introduction

1.1 Problem statement

After an earthquake, a cyclone or a flood, the ground mobile network in the affected area often fails: towers lose power, backhaul cables break and buildings collapse. Survivors cannot call for help and rescue teams cannot coordinate. A drone (unmanned aerial vehicle, UAV) carrying a small base station can act as a temporary tower. Three things make this hard:

1. the drone's battery lasts minutes, not hours, and flying (not the radio) dominates its energy use;
2. the air-to-ground signal depends on the drone's height and position, and collapsed buildings block some links;
3. survivors move, sometimes suddenly, and one drone must share its bandwidth and power among hundreds of people with different needs.

The planning question is therefore: **where should the drone fly, how high, and how should it share its radio resources over time so that as many survivors as possible stay connected before the battery runs out?**

1.2 Motivation

- Restoring a basic connection quickly matters more in a disaster than giving already-served users a fast one. This defines our main metric, **coverage**: the share of users who receive at least the minimum data rate of their class.
- Non-orthogonal multiple access (**NOMA**) lets two users share the same frequency slice at the same time by splitting power, so a power- and bandwidth-limited drone can serve more people. The guide's original idea was a moving base station using NOMA.
- Rescue planners cannot rehearse drone strategies on real disasters. A **simulation-based planning tool** lets strategies be compared fairly in software before any flight, saving hardware, battery and flight time. We deliberately do not call it a digital twin: that term needs live or replayed state data, which this project does not have.
- Reinforcement learning (RL) is often proposed for UAV control. We wanted to know honestly **when** a learned controller beats strong non-learning strategies and when it does not.

1.3 Objectives

1. Build an open, tested, reproducible simulator of a UAV base station serving moving survivor crowds with a NOMA downlink, with every physical parameter sourced or labelled as an engineering assumption.
2. Build strong non-learning pilots (baselines) and a fair evaluation protocol (same scenarios, paired statistics, fixed train/validation/test seeds, pass marks written down before results).
3. Measure NOMA against OMA inside the mission, for every pilot, including an equally optimised OMA.
4. Train RL pilots that learn by themselves from raw measurements and report where they win and lose.
5. Measure the room any anticipating controller could have (a clairvoyant upper bound) before spending effort on training.
6. Check that the simplified flight model does not distort the conclusions, by flying the same plans with a simulated quadcopter and autopilot.

7. Provide a dashboard so that non-specialists can set up a disaster, fly any pilot and compare them.

1.4 Hypotheses

Every experiment serves one of these hypotheses.

Table 1.1 Hypotheses

ID	Hypothesis	In plain words
H1	Adaptive UAV control beats a fixed UAV once crowds move	A drone that follows people does better than one parked in the middle
H2	RL beats strong heuristics (k-means, grid search, particle swarm) only when dynamics and battery limits make them short-sighted; report where it wins and where it does not	The AI should only win in hard, changing situations, and we must say where it loses
H3	NOMA pairing improves throughput and coverage over OMA at equal bandwidth and power	Sharing a slice between two people beats giving everyone a separate slice
H4 (stretch)	Coordinated multi-UAV control beats independent placement	Several drones working together beat drones acting alone

1.5 Scope

In scope: one UAV; 2 x 2 km and 1 x 1 km areas; 100-1,000 users in 2-5 crowds; six crowd-movement models and competing-site events; collapsed buildings as 2D rectangles; the Al-Hourani mean path loss with optional fading; the Zeng-Xu-Zhang energy model with a battery; NOMA (fixed and adaptive) and OMA downlinks; ten non-learning pilots, PPO and SAC agents; a clairvoyant planner; a simulated quadcopter with an autopilot; a Streamlit dashboard.

Out of scope: real hardware and flight tests; building heights, terrain and real maps (a real place from OpenStreetMap is planned only as a demonstration); the drone's own backhaul; interference from other cells; bit-error rate; more than one drone (H4 was not attempted). All results are conditional on the model: they may be used to compare strategies under the stated assumptions, not to predict absolute coverage in a real disaster.

1.6 Organisation of the report

Chapter 2 surveys the literature by theme. Chapter 3 describes the system design. Chapter 4 covers the implementation, testing and reproducibility. Chapter 5 presents the results, hypothesis by hypothesis. Chapter 6 discusses where learning helps, the limitations and threats to validity. Chapter 7 concludes and lists future work.

Chapter 2. Literature survey

The survey is organised by the six themes the project touches. The full matrix with links, and what we took from each paper, is in `docs/report/literature.md`. **Every entry still needs a person to check it against the original** (marked VERIFY there); of the two papers used for parameters, only [2] was read in full.

2.1 UAV base-station placement

A UAV base station's coverage depends on its 3D position. Bor-Yaliniz, El-Keyi and Yanikomeroglu [5] formulate 3D placement of one aerial base station as a mixed-integer non-linear problem and give an efficient numerical solution. Alzenad et al. [6] place a UAV base station to cover the most users with the least transmit power. Lyu, Zeng, Zhang and Lim [7] place several UAV-mounted base stations along a spiral to cover all ground terminals with as few drones as possible. Mozaffari et al. [4] survey placement, trajectory, energy and channel modelling for UAVs in wireless networks. These works optimise placement for a known, mostly static user set; our search pilots B3, B4 and B6 are simple members of this family (grid search, particle swarm [31]), used as strong baselines rather than as contributions.

2.2 The air-to-ground channel

Al-Hourani, Kandeepan and Lardner [1] give a closed-form probability of line of sight between a low-altitude platform and a ground user as a function of the elevation angle, and show that an optimal altitude exists for maximum coverage. The mean path loss is the free-space loss plus an excess loss weighted by the line-of-sight probability. We use this model with a = 9.61, b = 0.16, eta_LoS = 1 dB and eta_NLoS = 20 dB at 2 GHz. **The original letter was not read** (paywalled); the formulas and parameter values were checked in two secondary papers that cite it [27], [28], which list them without naming the environment, so the label "urban" is unconfirmed.

2.3 Rotary-wing energy

Zeng and Zhang [3] model the propulsion energy of a fixed-wing UAV and propose a circular trajectory with optimised radius and speed for energy efficiency. Zeng, Xu and Zhang [2] derive the propulsion power of a rotary-wing UAV as a function of speed: blade profile power, induced power and parasite power. Its key consequence for us is that moderate forward flight costs less than hovering. We verified the formula and constants (delta, k, rho, s) in [2] and derived our own small 20 N drone from it (hover 168.49 W); the fuselage drag ratio d0 = 0.6 is unverified ([2] uses 0.3).

2.4 NOMA and UAV-NOMA

Saito et al. [8] proposed power-domain NOMA with successive interference cancellation (SIC) for cellular radio access. Ding, Fan and Poor [9] show that with fixed power allocation, pairing users with very different channels enlarges NOMA's gain over OMA, and they study cognitive-radio-inspired NOMA in which the weak user's rate is protected. Liu et al. [10] present a framework for UAV communications based on NOMA. Cui et al. [11] jointly design the UAV trajectory and resource allocation for OMA and NOMA under max-min fairness. Chen et al. [12] allocate power in a NOMA-UAV network flying a circular trajectory, with a secrecy constraint. Our fixed scheme (strongest paired with weakest, 0.8/0.2 split) follows the fixed-allocation idea of [9]; our adaptive scheme gives each pair its minimum power in closed form, which is close in spirit to the rate-protecting allocation of [9].

2.5 Reinforcement learning for UAV networks

Deep RL has been applied to UAV base stations many times. Ghanavi et al. [16] use Q-learning to place an aerial base station for moving users. Liu, Liu, Chen and Hanzo [15] design trajectories and power control for multi-UAV networks with machine learning. Zhong et al. [13] combine

k-means with a deep Q-network to control the 3D trajectory and power of a UAV offloading NOMA users. AdaptSky [14] uses deep RL for joint NOMA power allocation and UAV placement. Hu et al. [17] use meta-RL so that an energy-limited UAV adapts its trajectory quickly to new demand. Rizvi and Boyle [18] use multi-agent RL with action masking for UAV mobile communications, and Wang et al. [19] split beamforming (fast, graph network) from trajectory (slow, multi-agent RL) in a two-layer design. **So the method is not new: we cannot claim to be the first to apply RL to UAV-NOMA.** In the papers we looked at, RL is usually compared with random, circular or OMA references; comparisons against strong search pilots on the same scenarios with paired statistics were rarer (to be checked across the full matrix).

The learners we use are PPO [20] and SAC [21] from Stable-Baselines3 [23], in a Gymnasium environment [24]. The privileged ("asymmetric") critic that sees the true state during training only follows Pinto et al. [22].

2.6 Evaluation practice

Henderson et al. [25] showed that deep-RL results vary strongly with random seeds and implementation details, and that few seeds can produce misleading conclusions. Agarwal et al. [26] recommend interval estimates and robust aggregate metrics rather than point scores. We follow both: fixed seed splits, several training runs, paired confidence intervals, a Holm correction [29] where many comparisons are made, Jain's fairness index [30] as a secondary metric, and pass marks written down before results are seen.

2.7 Literature matrix

Table 2.1 Literature matrix (short form; details and links in literature.md; every entry: VERIFY)

Ref.	Theme	Main idea	What we take or compare
[1] Al-Hourani 2014	Channel	LoS probability vs elevation; optimal altitude	Our channel model and parameters
[2] Zeng, Xu, Zhang 2019	Energy	Rotary-wing propulsion power	Our energy model; forward flight cheaper than hover
[3] Zeng, Zhang 2017	Energy	Fixed-wing energy; circular trajectory	Motivation for the circling pilot B7
[4] Mozaffari 2019	Survey	UAVs in wireless networks	Problem framing
[5] Bor-Yaliniz 2016	Placement	3D placement as an optimisation	Search pilots as baselines
[6] Alzenad 2017	Placement	Energy-efficient maximal coverage placement	Coverage as the objective
[7] Lyu 2017	Placement	Spiral placement of several UAV base stations	Multi-UAV future work (H4)
[8] Saito 2013	NOMA	Power-domain NOMA with SIC	NOMA principle
[9] Ding, Fan, Poor 2016	NOMA	User pairing; fixed vs rate-protecting power	Extreme pairing; adaptive split
[10] Liu 2019	UAV-NOMA	Framework for NOMA-based UAV networks	Context for H3
[11] Cui 2019	UAV-NOMA	Joint trajectory and resource allocation, OMA vs NOMA	Fair OMA reference
[12] Chen 2020	UAV-NOMA	Power allocation on a circular trajectory	Circling with NOMA
[13] Zhong 2020	RL + NOMA	k-means + DQN for trajectory and power	Shows RL-for-UAV-NOMA is published
[14] Benfaid 2021	RL + NOMA	DRL for NOMA power and placement	Same
[15] Liu 2018	RL	Multi-UAV trajectory and power control by ML	Same
[16] Ghanavi 2018	RL	Q-learning placement for moving users	Closest to our G5 task
[17] Hu 2020	RL	Meta-RL trajectories for an energy-limited UAV	Adapting to new demand
[18] Rizvi, Boyle 2023	RL	Multi-agent RL with action masking	Action design; H4
[19] Wang 2026	RL	Two-layer, two-timescale RL for multi-UAV	Supports our time-scale split
[20] Schulman 2017	Method	PPO	Our main learner
[21] Haarnoja 2018	Method	SAC	Our second learner
[22] Pinto 2017	Method	Asymmetric actor-critic	Privileged critic (training only)
[23] Raffin 2021	Tool	Stable-Baselines3	Implementation of PPO and SAC
[25] Henderson 2018	Evaluation	Seed variance in deep RL	Several runs per version
[26] Agarwal 2021	Evaluation	Interval estimates for RL results	Confidence intervals, pass marks

2.8 Gap and positioning

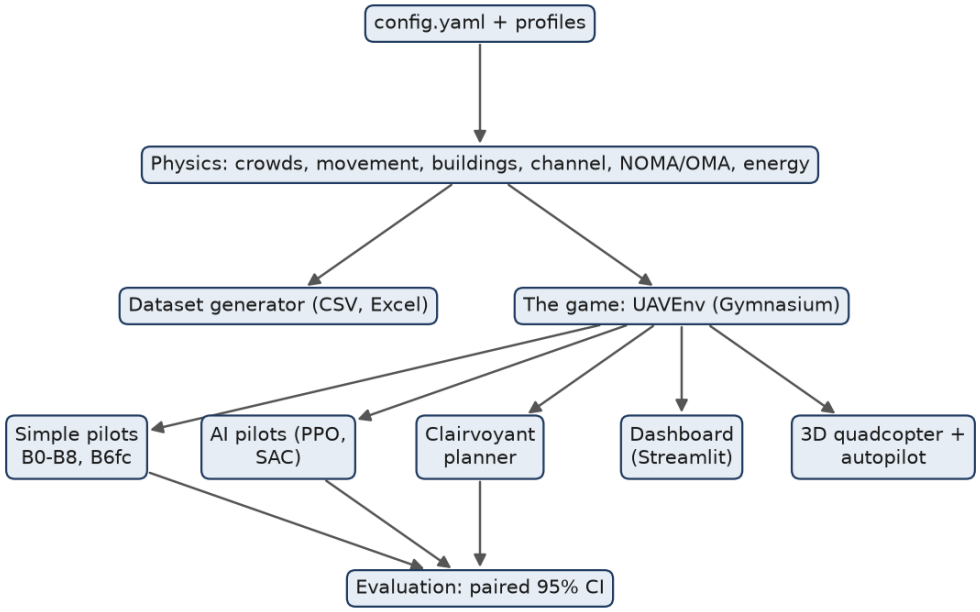
From the survey, the defensible contribution is not a new algorithm but **an open, tested simulator and evaluation protocol for UAV-NOMA planning in disaster scenarios, a comparison of learning against strong non-learning pilots across regimes, and an analysis of when learning helps**, together with specific findings: most of a search pilot's lead can come from choosing the altitude; forward flight costs less energy than hovering; NOMA's gain measured inside the mission for every pilot; and a method (clairvoyant headroom) for deciding whether a scenario leaves any room for a learning controller at all. Two further contributions are in progress: an anticipation rule that predicts that room from distance, speed and timing, and a design rule for how much phone location information a planner needs (D-56).

Chapter 3. System design

3.1 Architecture

The design principle is **one physics code path**: the dataset generator, the game (Gymnasium environment), every pilot, the AI, the dashboard and the 3D quadcopter all call the same functions in `make_dataset.py`, so they cannot disagree. Everything below the game is plain Python and NumPy with no user-interface or RL dependency, and every setting lives in `config.yaml`.

Figure 3.1 System architecture: one physics engine, one game, many pilots



3.2 Scenarios

A scenario is one invented disaster area identified by a seed; the same seed always gives the same crowd, buildings and movement. Each seed draws the number of users (100, 300, 600 or 1,000), 2-5 clusters with radius 50-200 m, 5-10% scattered users, the movement model and the buildings. Seeds are split and never mixed: **train 0-99, validation 100-119, test 120-139**; the RL agents later trained on fresh seeds 10,000-19,999. Named profiles change only what they name.

Table 3.1 Scenario profiles

Profile	Changes from the default (1 x 1 km, 30 dBm, 0-5 buildings, five movement models, 10 Wh)
`lowpower`	20 dBm
`bigarea`	2 x 2 km
`hard`	2 x 2 km, 20 dBm, 3-8 buildings of 60-200 m, only hotspot jump / surge / group drift / attraction, 16 Wh
`rl_static`	as `hard` but the crowd stands still (RL curriculum step 1)
`sites_d{10,15,20}_t{0,60,180}`	2-3 sites 1.0-2.0 km apart; 50-70% of the demand switches from site A to B at 60-150 s, at once or over 1 or 3 minutes; G5 settings inside

Every person belongs to a class that sets the minimum rate needed to count as served.

Table 3.2 User classes and minimum rates (engineering assumptions, not yet signed off)

Class	Share	Minimum rate
rescue	10%	200 kbps
video	20%	500 kbps
voice	30%	50 kbps
iot	20%	10 kbps
normal	20%	100 kbps

Movement. Six models move every person each second for 300 s, vectorised over users: static; random waypoint (0.5-1.5 m/s); group drift; attraction to 1-2 shelters; hotspot jump (one crowd relocates at 3 m/s at a random moment between 60 and 180 s); surge (everyone runs to one point at 3 m/s). In the competing-sites profiles an activity mask switches users off at site A and equally many on at site B, so crowd size, class mix and bandwidth stay the same.

Collapsed buildings. Axis-aligned rectangles; a drone-user link whose 2D segment crosses one gets 15 dB extra loss (engineering assumption, range 10-25 dB), tested with the vectorised slab (Liang-Barsky) method.

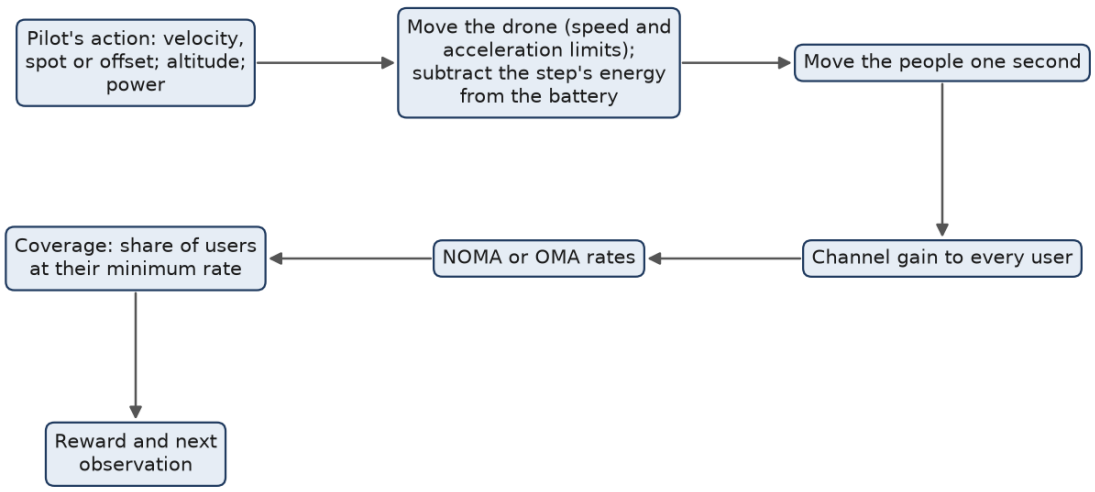
3.3 The radio channel

For a drone at (X, Y, H) and a user at (x, y), following Al-Hourani et al. [1]:

```
d      = sqrt((x - X)^2 + (y - Y)^2 + H^2)
theta  = (180 / pi) * arcsin(H / d)                          elevation angle in degrees
P_LoS  = 1 / (1 + a exp(-b (theta - a)))                      a = 9.61, b = 0.16
PL_dB  = FSPL(d, 2 GHz) + P_LoS * 1 dB + (1 - P_LoS) * 20 dB   (+ 15 dB if a building blocks the link)
noise  = -174 dBm/Hz + 7 dB noise figure, over each user's bandwidth
```

Optional fading (Rician K = 10 dB for LoS, Rayleigh for NLoS) is off for the dataset and the baselines. Bandwidth scales with the crowd: 50 kHz per user.

Figure 3.2 One second of the game



3.4 NOMA and OMA

Plain words. OMA gives every person a separate slice of the bandwidth. NOMA puts two people on the same slice: the weak user gets most of the power and reads its own message treating the other as noise; the strong user first decodes and removes the weak user's message (SIC) and then reads its own.

Four access schemes, all with the same total bandwidth B and power P (N users):

1. `oma`: FDMA, each user gets B/N and P/N.
2. `noma` (fixed): users sorted by channel gain, strongest paired with weakest; each pair gets 2B/N and 2P/N, split 0.8 to the weak user and 0.2 to the strong user (formulas below).
3. `oma_opt`: each user gets the minimum power for its rate, p_i = (2^(R_i/b) - 1) N0 b / g_i; users are admitted cheapest first while the total stays within P; leftover power is scaled up evenly.
4. `noma_opt` (adaptive NOMA): the same cheapest-first admission, then the admitted users are paired strongest with weakest on a double slice and each pair gets its minimum NOMA power in closed form, with an imperfect-SIC residual eps (formulas below). If NOMA would cost more than serving the pair separately (possible only with eps > 0), the pair uses OMA inside its slot. The largest number of users that fits is found by binary search; options `sic_eps` (0 or 0.05) and `rescue_first`.

```
fixed NOMA:     SINR_weak   = 0.8 p g_w / (0.2 p g_w + N0)
                SINR_strong = 0.2 p g_s / N0                  (after SIC)
                R = b log2(1 + SINR)
adaptive NOMA:  p_s = gamma_s N (eps gamma_w / g_w + 1 / g_s) / (1 - eps gamma_s gamma_w)
                p_w = gamma_w (p_s + N / g_w),      gamma = 2^(R / 2b) - 1
```

From G5 on, the project uses `noma_opt` with eps = 0.05 and rescue users admitted first, and always reports `oma_opt` next to it so that H3 stays a fair test.

3.5 Energy and battery

Rotary-wing propulsion power after Zeng, Xu and Zhang [2]:

```
P(V) = P0 (1 + 3 V^2 / Utip^2) + Pi sqrt( sqrt(1 + V^4 / (4 v0^4)) - V^2 / (2 v0^2) ) + 0.5 d0 rho s A V^3
hover P(0) = P0 + Pi = 79.86 + 88.63 = 168.49 W
energy per 1 s step = P(V) + W max(climb rate, 0) + P_tx / 0.4 + 5 W
```

For our 20 N drone: Utip = 120 m/s, v0 = 4.03 m/s, d0 = 0.6, rho = 1.225 kg/m^3, s = 0.05, A = 0.503 m^2. Flying at about 10 m/s costs 133.5 J/s against 176.0 J/s hovering (30 dBm, including radio and electronics). An acceleration limit (4 m/s^2 horizontal) and a load-factor term for turning and braking (our own extension, labelled as such) close a "shaking" exploit in which a drone could stay in place at forward-flight power.

3.6 The game (Gymnasium environment)

`UAVEnv` plays one scenario second by second for 300 s or until the battery is empty. The drone starts at the map centre at 150 m, flies at most 15 m/s horizontally and 5 m/s vertically, between 50 and 300 m. The reward is the coverage of the second; each reward part is logged separately so that reward hacking can be spotted. The observation contains only what a real drone could measure: its position, velocity, battery and time, its previous coverage, throughput and fairness, and coarse 10 x 10 maps of where users and rescue users are. Later versions added an uncapped map, the number of connected users, the drone's own history and, in the competing-sites cells, the operator's announcement of the event. Pilots can control velocity (`xy`), fly to a spot (`goto`), or move by an offset (`moveby`, 0 = stay), and the AI decides every 10 s.

The rule about the AI (owner's decision): the AI learns by itself from raw measurements and the coverage score. No hints, no menu of candidate spots, no imitation of the baselines, no privileged information when flying. A critic that sees the true state during training only was allowed later and is disclosed wherever used.

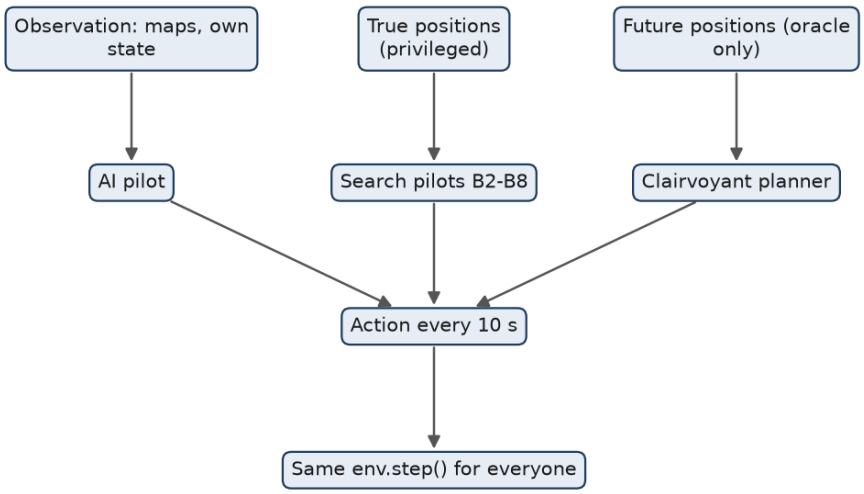
3.7 Pilots

The baselines see the true user positions (privileged information); the AI sees only maps. That makes them a strong test.

Table 3.3 The pilots

ID	Name	What it does
B0	Fixed	Parks at the map centre at 150 m, full power
B1	Follow the crowd	Flies toward the centre of the biggest true cluster
B2	Weighted k-means	Every 10 s: k = 3 clusters (rescue users count 3x), flies to the heaviest
B3	Grid search	Every 10 s: scores a grid of spots at 150 m, flies to the best
B4	Particle swarm (PSO)	Every 10 s: searches position, altitude and power
B5	Energy-aware	Like B3 but moves only if the gain pays for the energy; behaves like B3 because moving is cheaper than hovering
B6	Grid search + altitude	Like B3 with five altitudes per spot
B7	Circling	Chooses spots like B6 but circles them (50 m) at the minimum-power speed
B8	Battery rule	Like B6; circles once the battery would not last
B6fc	Forecasting	Like B6, also using the announced event schedule (competing sites)
B3obs, B3obsU, B3obsUO	Information ladder	B3's search using only the AI's maps (diagnostic)
Clairvoyant	Upper bound	Knows the crowd's whole future; dynamic programming over B6's grid
AI	PPO / SAC	Neural network trained by RL; sees only raw measurements

Figure 3.3 How each pilot reaches the game



3.8 The AI

PPO [20] (and SAC [21] for comparison) from Stable-Baselines3 [23]. Two network types: a two-layer MLP (64 units) and a small CNN that reads the maps as a picture with coordinate channels. The best still-crowd version (E0) uses a dictionary observation with uncapped maps, the crowd size and the last 4 positions and coverages, fresh training scenarios and a planning horizon gamma = 0.95 matched to 30 decisions per mission. Training runs on a laptop CPU (about 1,000 simulated seconds per second when plugged in) and, for G5, on Kaggle.

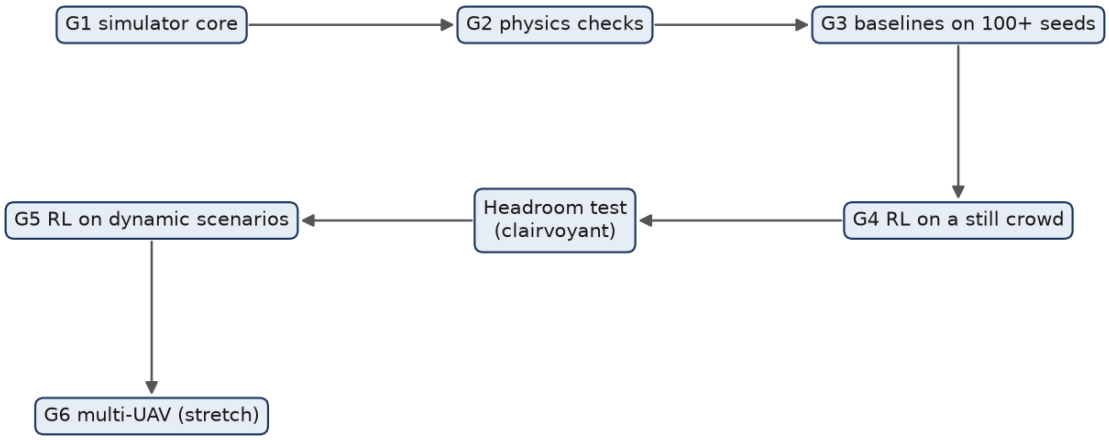
3.9 The evaluation protocol

- Main metric **coverage_full**: coverage averaged over the 300 s mission, counting zero after the battery dies.
- Mean and 95% confidence interval over scenarios; **paired differences** per scenario (pilot A minus pilot B on the same scenario) so that "this scenario was easy for everyone" cancels; a *

marks a CI that excludes zero.

- Holm correction where a table makes many comparisons.
- Pass marks fixed **before** results are seen (G4: D-43; competing sites: D-52; G5: D-55); test seeds read once at the end.
- Gates: a layer starts only after the previous gate passes.

Figure 3.4 The evaluation protocol and the gates

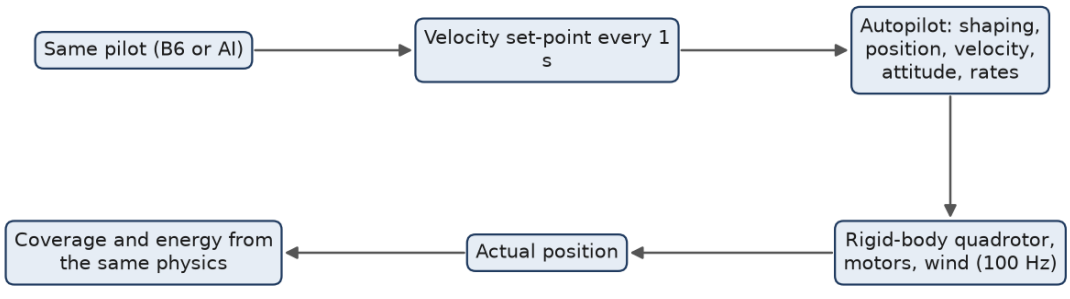


3.10 The dashboard and the 3D quadcopter

Dashboard. A Streamlit web page on the same engine, with pages to see the showcase, fly a disaster with any pilot, compare pilots, get a plan with battery advice, and (since Stage 5) watch a 3D flight. Its numbers equal the experiments' because it calls the same code; 16 tests check this without Streamlit.

Simulated quadcopter (Stage 5). In the game the drone moves like a chess piece. In Stage 5 every pilot's velocity command instead becomes the set-point of an autopilot (like ArduPilot GUIDED velocity mode) flying a rigid-body quadrotor model of the same 2.04 kg drone: thrust and attitude dynamics, a first-order motor lag, rotor and fuselage drag, a 30 degree tilt limit, a thrust-to-weight ratio of 2, a geofence, and wind with seeded gusts, integrated at 100 Hz. Energy is still computed by the Zeng model, from the airspeed, climb and acceleration of every integration step. The autopilot values quoted from ArduPilot and PX4 are marked as not verified.

Figure 3.5 The 3D quadcopter in the loop (sim-to-sim)



Chapter 4. Implementation

4.1 Tools

Python with NumPy, pandas, matplotlib, PyYAML, Gymnasium, pytest, xlsxwriter and openpyxl for the simulator and the baselines; Stable-Baselines3, PyTorch (CPU) and TensorBoard for the AI; Streamlit for the dashboard; ReportLab for the PDF documents. Development on Windows 11 (laptop CPU); continuous integration on GitHub with Python 3.11 and 3.12.

4.2 Code structure

Table 4.1 Code structure (`uav_dataset/`)

File	Role
`config.yaml`	Every setting and profile; values are changed here, never in the code
`make_dataset.py`	Physics, scenarios, movement, buildings, channel, NOMA/OMA access schemes, energy, dataset tables
`uav_env.py`	The Gymnasium game
`baselines.py`	Pilots B0-B8, B6fc and the information-ladder pilots
`run_baselines.py`, `evalstats.py`	Runner and statistics (mean +/- 95% CI, paired differences)
`rl_utils.py`, `rl_net.py`, `rl_train.py`, `rl_eval.py`	Control modes, decision hold, CNN, training and evaluation
`oracle.py`	Clairvoyant planner (dynamic programming)
`sites_exp.py`, `g5_eval.py`	Competing-sites experiments and the G5 evaluation
`noma_vs_oma.py`, `noma_picture.py`, `access_benchmark.py`	H3 experiments and pictures
`accel_loiter_exp.py`	Circling experiment
`quadsim.py`, `sim2sim_exp.py`, `viewer3d.html`	Simulated quadcopter, sim-to-sim check, 3D view
`dashboard.py`, `dashboard_core.py`	Streamlit dashboard and its tested logic
`view_run.py`, `compare_run.py`	Single-pilot GIFs and multi-pilot comparison pictures
`test_*.py`	Automated tests

4.3 Testing and continuous integration

- **399 automated tests** (2026-10-23), covering the NOMA maths (including brute force on small crowds and an independent re-derivation of the closed form, exact to 1e-15 over 2,000 random pairs), the mobility models, the obstacle geometry, the energy model, the Gymnasium checker, every pilot, the statistics, the clairvoyant planner (equal to brute force on a small grid), the privileged-critic guard (the flying network's output is bit-identical when only the privileged data changes), the dashboard logic and the quadcopter (26 tests).
- Every pull request runs ruff lint, pytest on Python 3.11 and 3.12 with an 80% coverage gate on the core modules, a smoke run of the generator, and a separate RL job with a short PPO training.
- `main` is protected: pull requests, green CI and code-owner review.
- Bugs caught this way include the fading flag not reaching the channel, a missing dependency, a policy adapter used before `reset()`, and a "shaking" energy exploit found by a reviewer and closed with an acceleration limit.

4.4 Reproducibility

- Seeded random generators only; the same seed gives the same scenario; a training run killed by a session restart was re-run and printed the same validation scores.
- New features are off by default, so every earlier number stays bit-exact (tested).

- Generated data is not committed but regenerated in about a minute from `config.yaml` and the seeds.
- Every model saves its settings next to it; results come only from committed configs and seeds.
- Every physical parameter has a value, unit, source or "engineering assumption", and a sensitivity range (`docs/assumptions.md`).
- A project book records every step, decision and result, with corrections added as new entries rather than rewrites.

Table 4.2 Dataset tables (version 3, 140 scenarios)

Table	Rows	One row is
scenarios	140	one scenario
obstacles	341	one collapsed building
users	74,900	one person at t = 0
trajectory	823,900	one person at one 30 s snapshot
sweep	813,120	one drone position, altitude and power in one scenario

Chapter 5. Results

All numbers are coverage (share of users served) unless stated; values are means over scenarios with 95% confidence intervals where given; * marks a paired difference whose CI excludes zero. Each section states which radio scheme was used, so that NOMA stays visible in every result.

5.1 Physics checks (Gate G2)

An optimal altitude exists for every user class and moves lower as the rate target rises: at 0 dBm the best altitude is about 500 m for iot (10 kbps), 200 m for voice, 100 m for normal and 50 m for rescue and video users (fixed NOMA, dataset v2). Altitudes above 300 m are diagnostic only (the drone flies at 50-300 m).

Figure 5.1: Four training scenarios: crowds, scattered users, rescue users circled

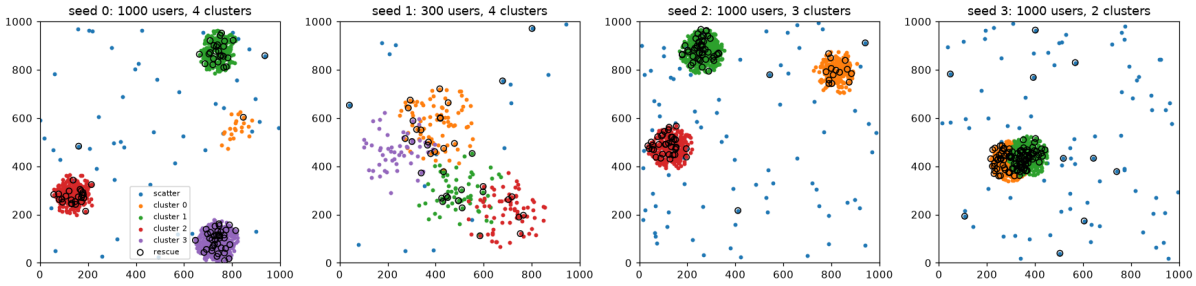


Figure: users_preview_v3.png

Figure 5.2: Coverage against altitude for every user class

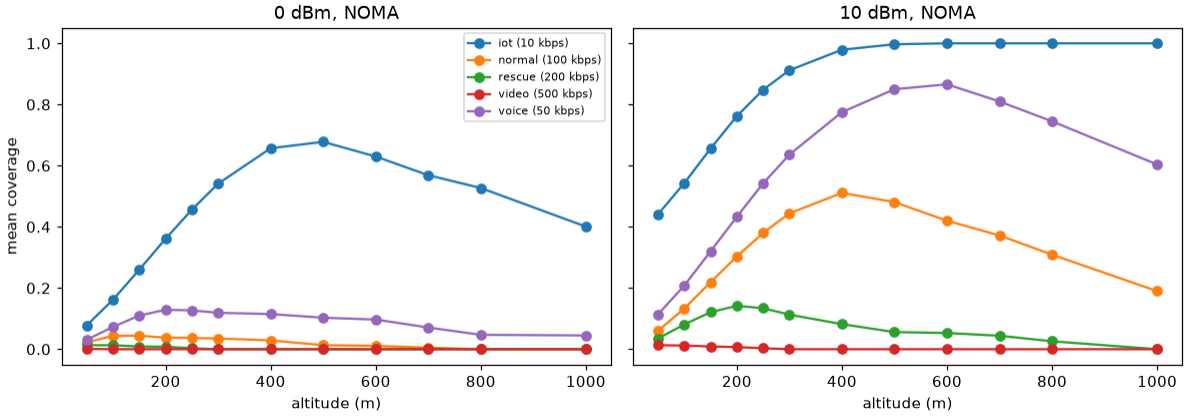


Figure: altitude_by_class_v2.png

5.2 H1: an adaptive drone beats a parked one

In the default 1 x 1 km scenarios with plenty of battery (16 Wh) a parked drone already covers 0.882 and the best pilot 0.914: little headroom. In the `hard` profile (2 x 2 km, 20 dBm, 3-8 buildings, sudden or drifting crowds, 16 Wh) the parked drone falls to 0.428 while the best pilots reach 0.740-0.741: **a 31-point gap, so H1 holds where the area is large and power scarce.** Neither low power alone nor a big map alone produces this gap; the combination does.

Table 5.1 H1: pilots on the `hard` *profile (100 scenarios, 16 Wh, all survive; fixed NOMA)*

Pilot	Coverage	Rescue users not served	Fairness
B0 fixed	0.428 +/- 0.042	0.846	0.504
B1 follow	0.627 +/- 0.027	0.535	0.538
B2 k-means	0.633 +/- 0.027	0.524	0.562
B3 grid	0.675 +/- 0.025	0.506	0.592
B4 PSO	0.740 +/- 0.023	0.417	0.709
B6 grid + altitude	0.741 +/- 0.023	0.419	0.715

Finding: most of B4's lead was altitude, not placement. B6 (grid search plus five altitudes) ties B4 (0.741 vs 0.740) and beats B3 by 0.066*.

Figure 5.3: Seven pilots on the hard profile

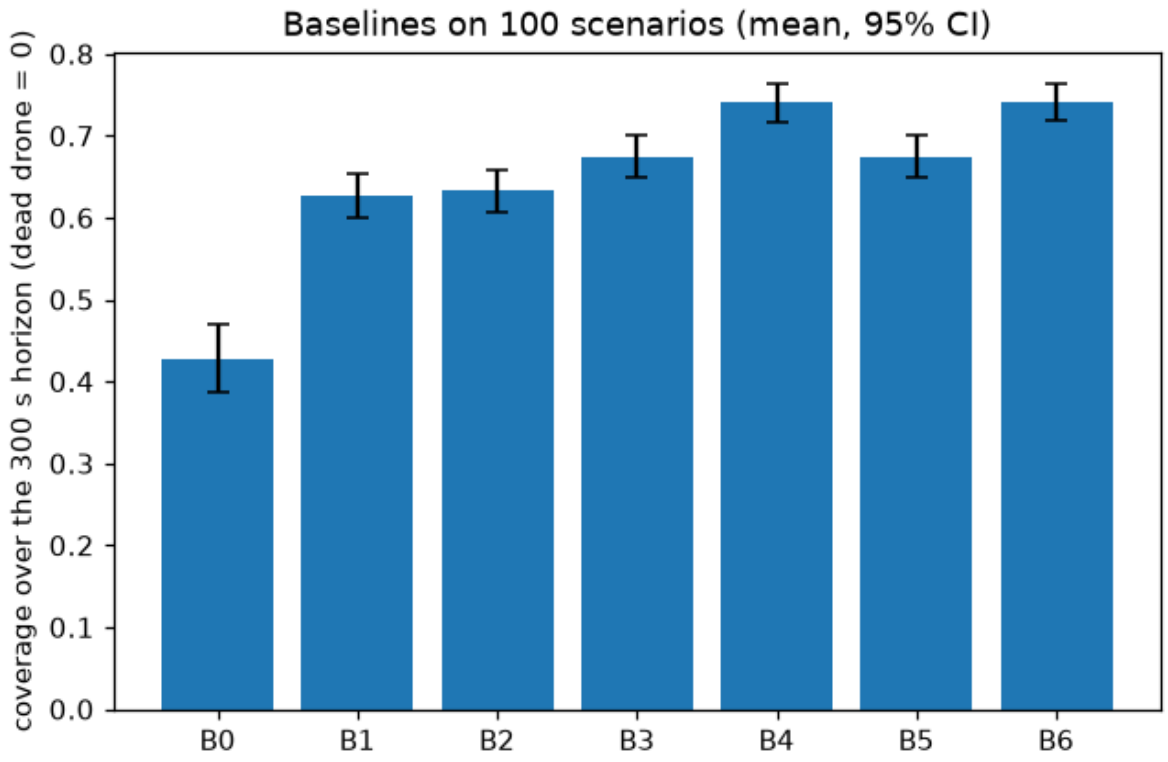


Figure: baselines_hard_7policies.png

Figure 5.4: Pilots on hard scenario 5 (600 users, 5 crowds, group drift): B0 0.49, B2 0.51, B3 0.55, B4 0.71, B6 0.71

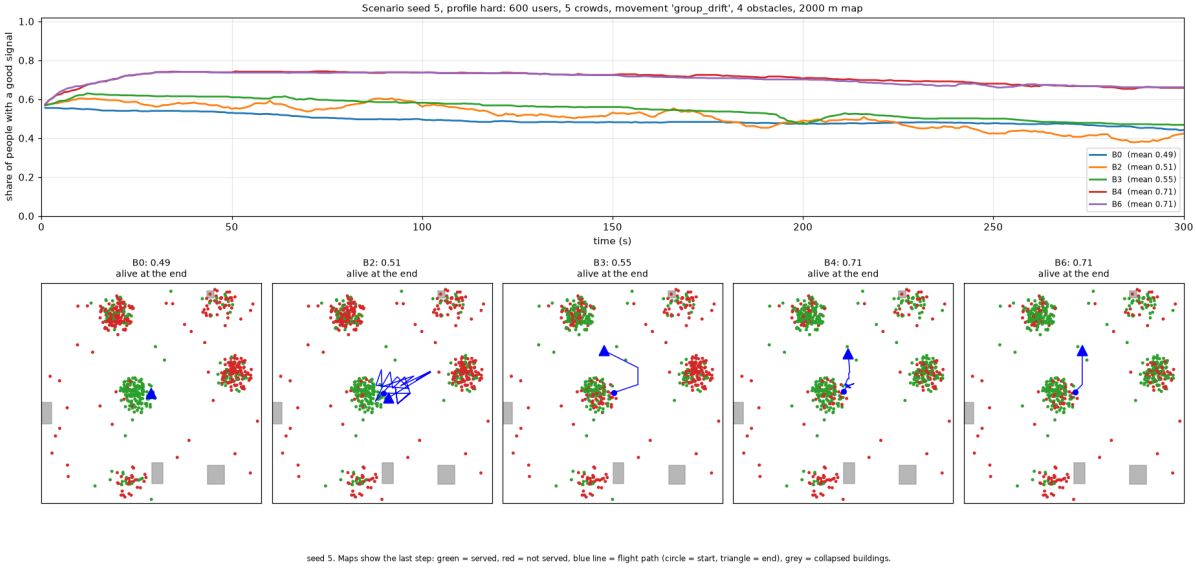


Figure: 1_hard_moving_crowds.png

5.3 H3: NOMA against OMA

Fixed NOMA (0.8/0.2 split) against OMA, inside the game. Every pilot played the same 100 scenarios under both schemes, with the search pilots evaluating spots under the scheme being tested.

Table 5.2 H3: fixed NOMA vs OMA in the game (paired NOMA minus OMA)

Profile	Pilot	Coverage NOMA / OMA / difference	Total rate (Mbps) NOMA / OMA
default, 30 dBm	B0	0.882 / 0.870 / +0.012*	275.1 / 228.8
default, 30 dBm	B3	0.909 / 0.929 / -0.020*	297.7 / 270.9
lowpower, 20 dBm	B0	0.797 / 0.745 / +0.052*	171.8 / 141.2
lowpower, 20 dBm	B3	0.865 / 0.808 / +0.058*	218.4 / 172.1
hard, 20 dBm	B2	0.633 / 0.568 / +0.065*	159.5 / 119.2
hard, 20 dBm	B3	0.675 / 0.616 / +0.059*	155.8 / 104.9

NOMA always gives more total rate (about 20-27% at 30 and 20 dBm) and serves 5-7 more people per 100 when power is scarce. At full power with a well-placed drone, OMA covers slightly more (1.7-2.0 fewer people per 100 with NOMA), and NOMA is less fair (Jain's index 0.10-0.20 lower).

Why fixed NOMA loses coverage at full power. With a fixed split the weak user always hears the strong user's signal, so SINR_weak < 0.8 / 0.2 = 4 however much power is used, and its rate is capped at (2 x 50 kHz) x log2(5) = **232 kbps**. A video user (500 kbps) in the weak half of a pair can never be served. This is the classic limitation of fixed power allocation and it motivated the adaptive scheme.

Figure 5.5: NOMA vs OMA in the game, hard profile

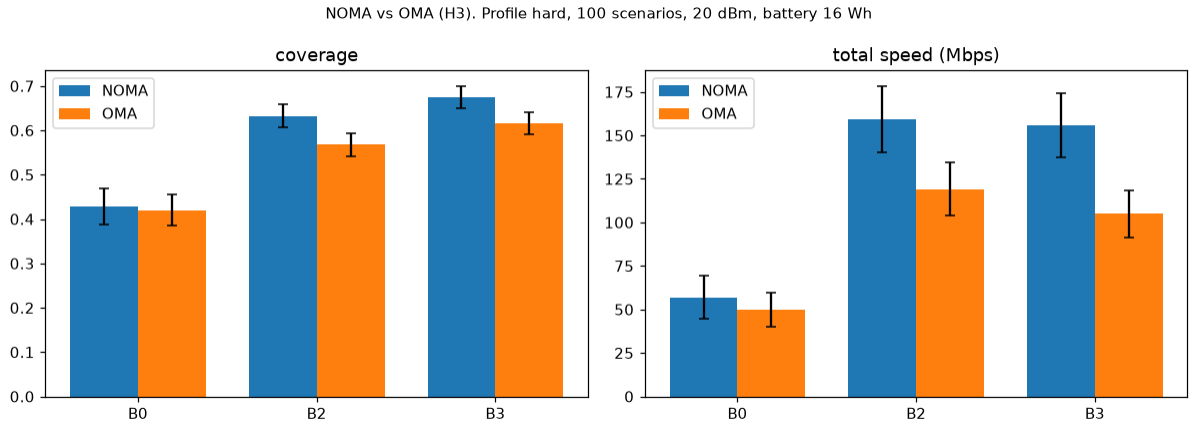


Figure: noma_vs_oma_in_game_hard_20dBm.png

Figure 5.6: NOMA pairs on the map and every user's rate (scenario 7, 1,000 users, 20 dBm): NOMA 0.81 vs OMA 0.76; the weak users sit on the 232 kbps ceiling

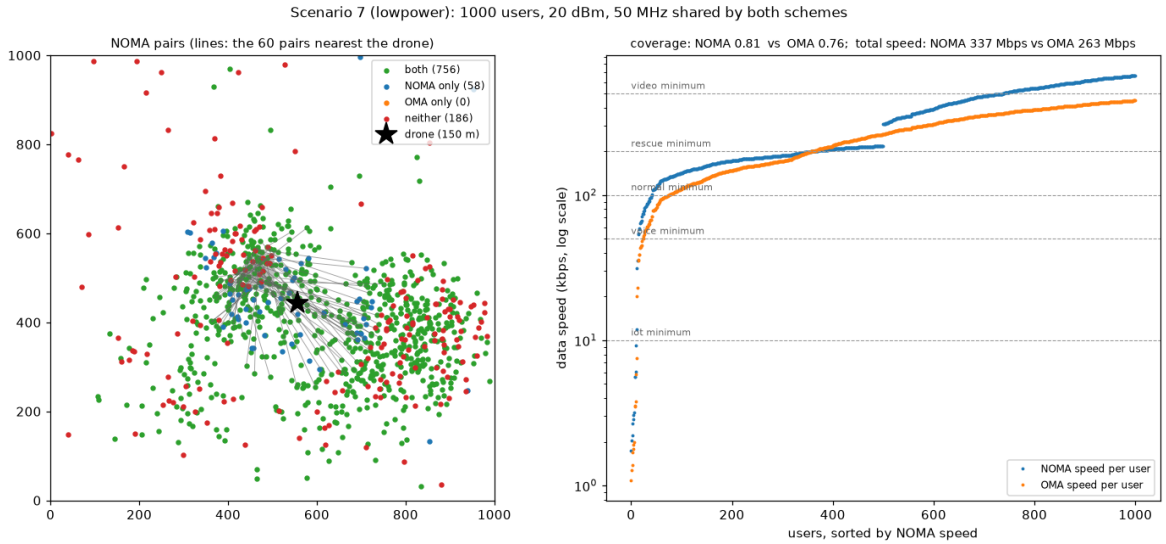


Figure: 7_noma_pairs_lowpower.png

Adaptive NOMA against equally optimised OMA. A fair test optimises both schemes the same way (Section 3.4).

*Table 5.3 H3: adaptive NOMA vs equally optimised OMA (100 scenarios, 16 Wh; * significant after Holm correction over 60 comparisons)*

Profile, pilot	OMA	NOMA fixed	OMA optimised	NOMA adaptive	Adaptive NOMA minus optimised OMA
lowpower, B0	0.745	0.797	0.893	0.944	+0.051*
lowpower, B2	0.769	0.835	0.924	0.962	+0.039*
lowpower, B3	0.808	0.865	0.945	0.983	+0.038*
hard, B0	0.420	0.428	0.658	0.690	+0.032*
hard, B2	0.568	0.633	0.777	0.811	+0.034*
hard, B3	0.616	0.675	0.808	0.849	+0.041*

- **Adaptive NOMA serves 3-5 more people per 100 than equally optimised OMA for every pilot**, with more total rate, better fairness and fewer unserved rescue users (B3 on lowpower: rescue outage 0.004 vs 0.030). The 232 kbps ceiling is gone.
- **Imperfect SIC** (eps = 0.05, hard): adaptive NOMA loses 0.014-0.024 coverage but still beats optimised OMA by +0.016 to +0.018*.

- **Rescue first** (hard): rescue outage falls sharply with adaptive NOMA (B3 0.203 to 0.117) for a small coverage loss (0.011-0.029); with optimised OMA the same rule costs 0.11-0.20 coverage.
- **The honest trade-off:** both optimised schemes serve far more people than the fixed ones (+0.12 to +0.26) but deliver about 55-65% less total rate and lower fairness than the fixed schemes. This is the price of the goal "serve as many people as possible", not of NOMA; the claim is "more people served, less total rate", not "better on every metric".

Figure 5.7: Four access schemes, hard profile

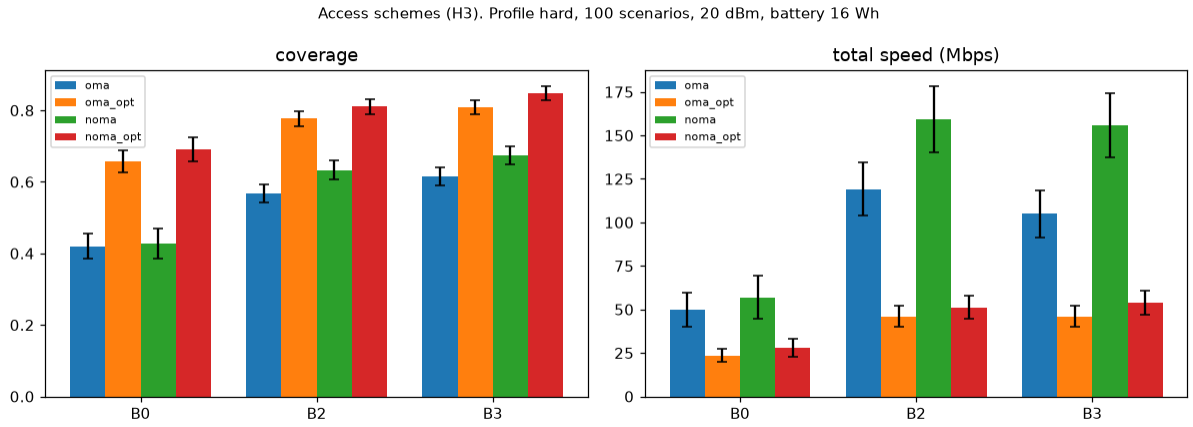


Figure: 8_access_schemes_hard.png

Figure 5.8: Four access schemes, low-power profile

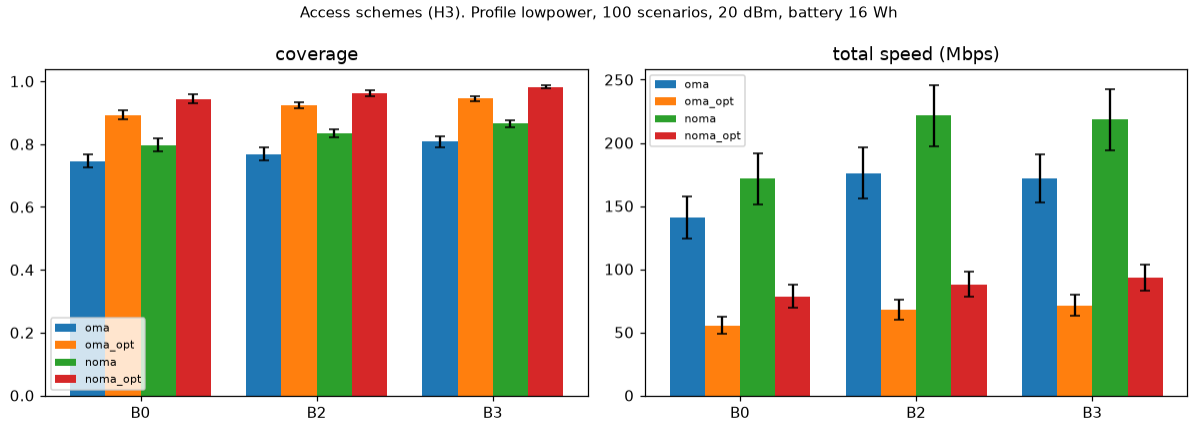


Figure: 9_access_schemes_lowpower.png

5.4 Energy: circling beats hovering on a tight battery

In the Zeng model the minimum-power speed is about 10.2 m/s (126.0 W against 168.5 W hovering). Pilot B7 chooses its spots like B6 but circles them at 9.8 m/s. With the acceleration limit and turning cost on, flying until the battery is empty (still crowds, 10 Wh, 50 scenarios):

Table 5.4 Circling vs hovering (fixed NOMA, full power)

Test	B6 (hovers)	B7 (circles, 50 m)	Difference
People-seconds served until the battery is empty	67,682	85,033	+25.6%*
Flight time	198 s	249 s	
Coverage while flying	0.697	0.696	same
5-minute mission, hard, 12 Wh	0.596 (never survives)	0.716	+0.120*
5-minute mission, hard, 13 Wh	0.652 (never survives)	0.730	+0.079*
5-minute mission, hard, 16 Wh	0.739	0.731	-0.009*

Figure 5.12: Circling vs hovering: 5-minute missions on a tight battery and flying until empty

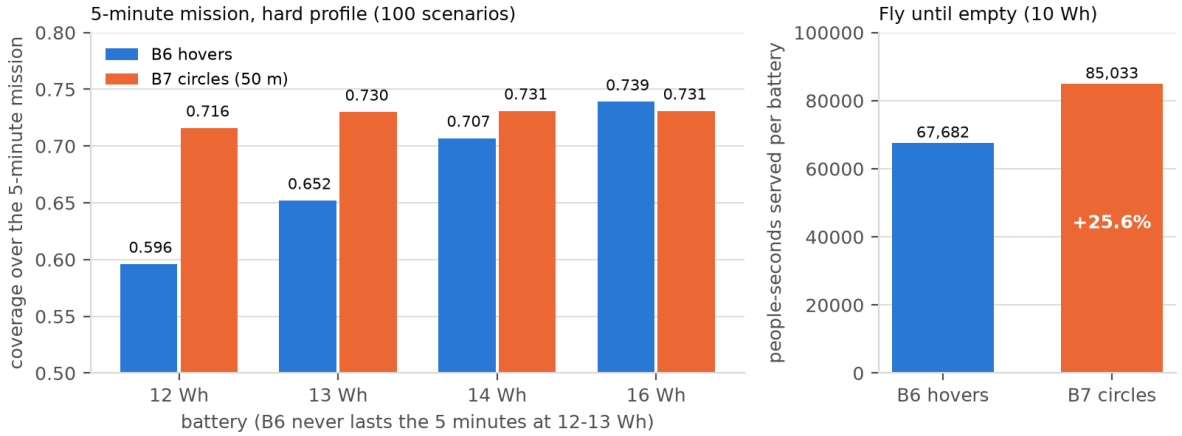


Figure: circling_vs_hovering.png

The gain stays between +21% and +31% when the unverified drag d0, the induced power, the acceleration limit or the radius are changed one at a time, so it does not depend on d0 = 0.6. A one-line rule (B8: circle once the battery would not last) captures the whole battery decision (12 Wh: 0.716; 13 Wh: 0.733), which means **a tight battery alone leaves nothing for an AI to win**. The turning-energy term is our own quasi-steady extension and is not validated against measurements.

5.5 Why the AI stalls on a still crowd: the information ladder

The search pilots score spots with every person's true position; the AI sees only coarse maps. To separate missing information from poor learning, three copies of B3's search were given less information.

Table 5.5 The information ladder (validation seeds 100-119, `rl_static`, fixed NOMA, paired with the AI E5a)

Pilot	What it knows	Coverage	vs the AI
B3	everything	0.659	+0.104*
B3obsUO	uncapped maps + buildings	0.629	+0.074*
B3obsU	uncapped maps	0.613	+0.059*
B3obs	exactly what the AI sees	0.605	+0.050*
B2 k-means	true positions	0.610	+0.055*
AI E5a	maps (learned)	0.555	
Random	nothing	0.430	-0.125*

About half of the AI's gap to B3 is missing information (0.054) and half is the AI not using what it sees (0.050). The AI is behind even on scenarios it trained on, so the problem is underfitting, not memorising. The key insight: B3 often parks **between** crowds so that one footprint reaches several of them; flying to the densest cell gives only 0.565, which is where every AI version stalled. The AI learned "go to the crowd", not "find the spot that reaches several crowds".

5.6 RL on a still crowd: Gate G4 (negative result)

Gate G4 asked the AI to beat random and match B2 on still crowds (`rl_static`, validation seeds, fixed NOMA). The pass mark was fixed before the final result: **the 3-run average must reach B2 - 0.03 = 0.580.**

Table 5.6 Gate G4 runs (validation, 20 scenarios)

Version	What changed	Coverage
First PPO	velocity control, decides every second	0.394 (below random 0.413)
`goto_hold10`	fly-to-spot control, decides every 10 s	0.525
E3	CNN map reader	0.549
E5a	CNN, 2.5x more training	0.555
E6 / E6b	exploration noise annealed	0.509 / 0.536
E0	uncapped maps, crowd size, own history, fresh scenarios, gamma 0.95 (3 runs)	**0.576 +/- 0.011**
E2	SAC	0.567
E3 (Track 2)	SAC + privileged critic (3 runs)	0.565 +/- 0.029
Bars	Random 0.430, B2 0.610, B3obs 0.605, B3 0.659	

Verdict: G4 not met (0.576 < 0.580). On a second unseen set (seeds 0-19) E0 averaged 0.519 against B2's 0.568 (-0.050*), so the gap is real. What worked: fly-to-spot controls and a 10-second rhythm (from below random to +0.095* above it), better inputs and fresh scenarios (0.555 to 0.576). What did not: a CNN, a finer map, longer training, annealed noise, SAC, and the privileged critic. The remaining fault is visible in Figure 5.11: the AI reaches the best spot at about 30 s and then flies on.

Figure 5.9: The first AI (velocity control) reaches the crowd and then flies off the map (seed 105: random 0.51, B0 0.53, B2 0.68, B3 0.78, PPO 0.40)

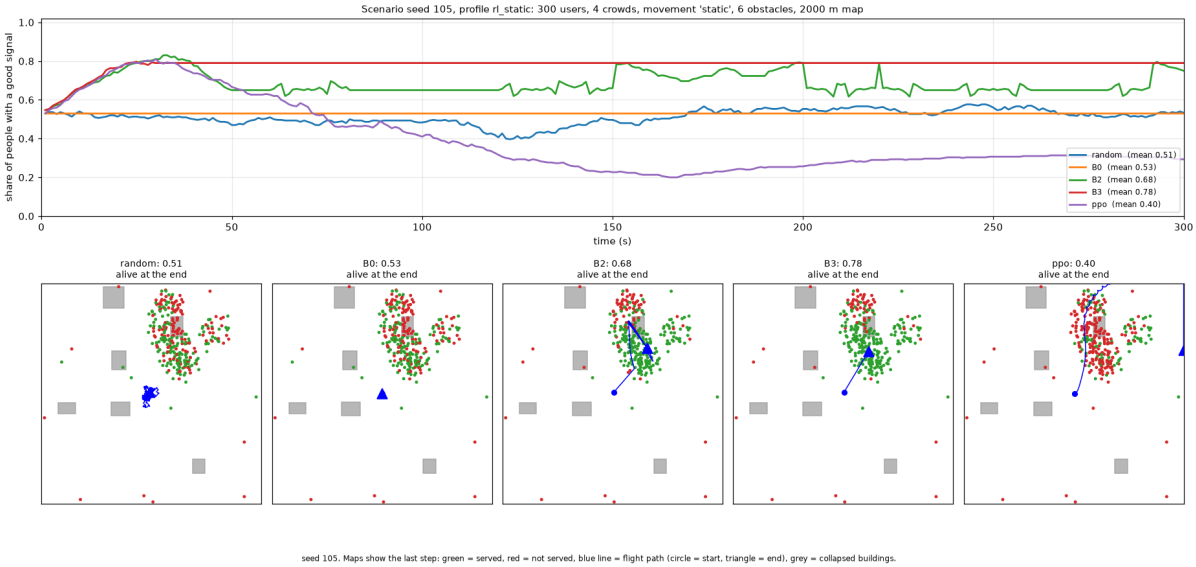


Figure: 2_still_crowd_AI_vs_pilots.png

Figure 5.10: Fly-to-spot control with a 10-second rhythm

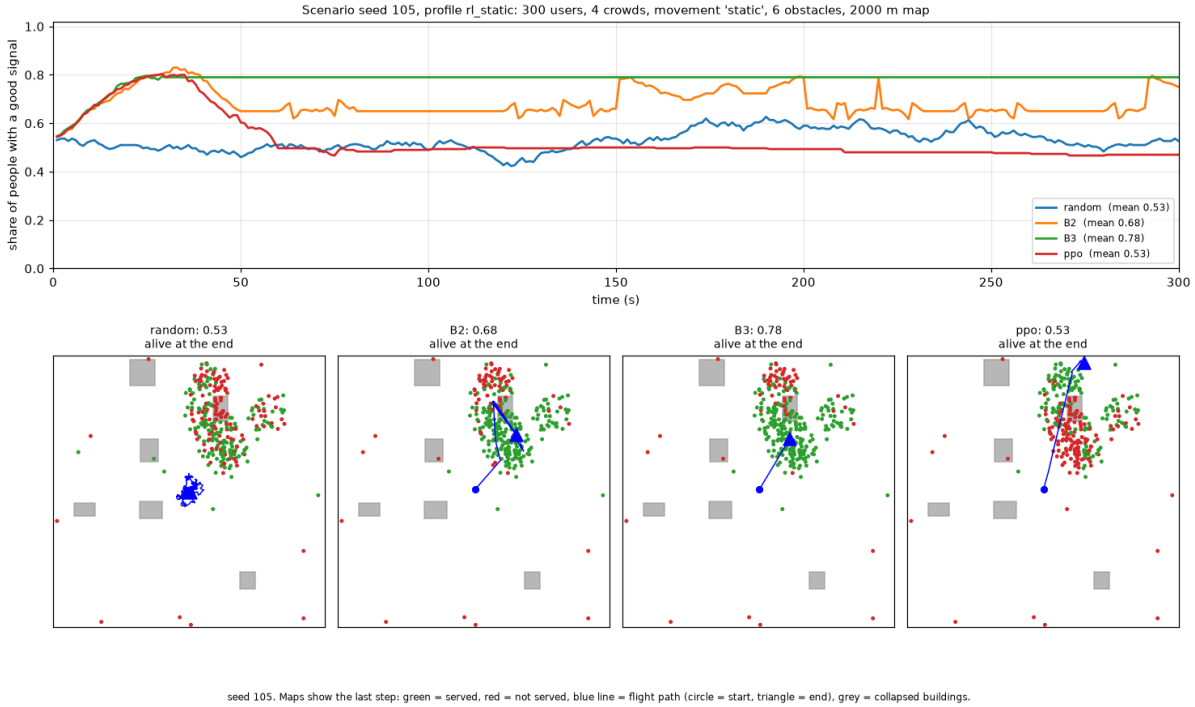


Figure: 5_goto_hold10.png

Figure 5.11: Best AI (E0) vs grid search B3 on scenario 105: the AI passes the best spot and settles at 0.66; B3 stays at 0.78

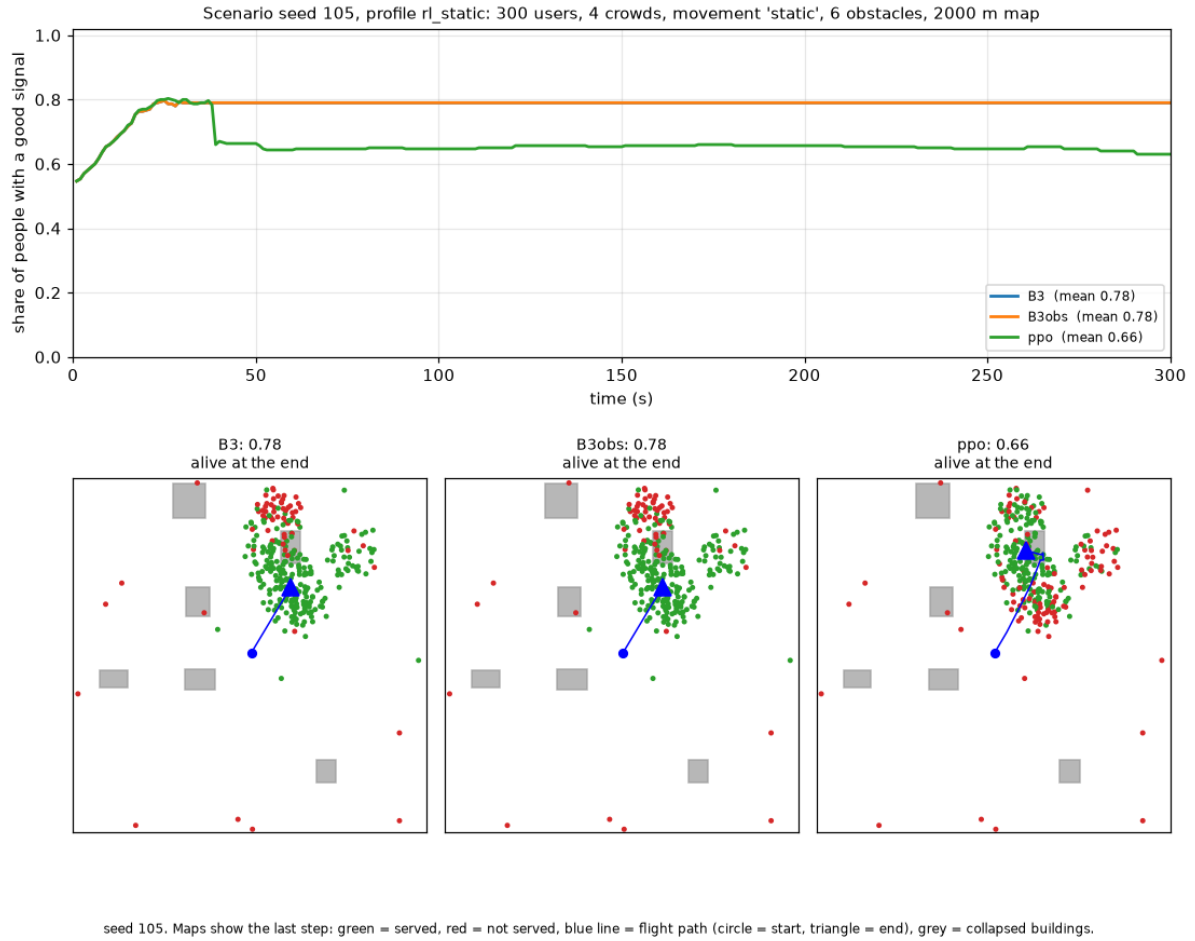


Figure: 10_e0_vs_b3_seed105.png

5.7 Is there room for an anticipating AI? Clairvoyant headroom and competing sites

H2 says an AI should win where it can anticipate. Before training on moving crowds we measured the prize: a clairvoyant pilot that is told the crowd's whole future plans the best possible flight by dynamic programming over B6's 980 spots, and its plan is flown in the real game. No AI can beat it, so its lead over B6 is the most anticipation could ever add. Pass mark fixed beforehand: at least 0.02 for some crowd type.

Table 5.7 Clairvoyant headroom on walking crowds (`hard`, 16 Wh, 120 scenarios, fixed NOMA, paired with B6)

Crowd type	B6	Clairvoyant	Gain
attraction (32)	0.776	0.778	+0.001
group drift (29)	0.745	0.747	+0.002
hotspot jump (26)	0.701	0.702	+0.001
surge (33)	0.748	0.750	+0.003
all (120)	0.745	0.746	+0.002*

Knowing the whole future is worth +0.002 (also +0.001 with adaptive NOMA): the drone (15 m/s, re-planning every 10 s) is much faster than walking crowds (0.5-3 m/s) and its footprint is hundreds of metres wide, so greedy re-planning keeps up. A pilot that looks only 30 s ahead even does worse than B6 (-0.011*). This saved weeks of training on scenarios where no AI could win.

Redesign: competing sites (pre-registered before any run). Survivors are at 2-3 sites 1.0-2.0 km apart, and half or more of the demand switches from site A to site B (a shelter opens, or a tower is repaired), at once or over 1 or 3 minutes. These runs use the G5 settings (adaptive NOMA, SIC residual 0.05, rescue first, acceleration limit).

Table 5.8 Room in the competing-sites designs (clairvoyant minus the best of B6, B7, B8; 12 scenarios per design; pass +0.02)

Sites apart (flight time)	Sudden switch	Over 1 minute	Over 3 minutes
1.0 km (67 s)	+0.025	+0.009	+0.005
1.5 km (100 s)	+0.072	+0.039	+0.020 (borderline)
2.0 km (133 s)	+0.108	+0.098	+0.056

7 of 9 designs passed on these design scenarios. **Room appears when the flight across is about as long as, or longer than, the switch.** As the pre-registered rules required, the designs were then re-measured on fresh scenarios, and a forecasting pilot was added to the bar: **B6fc** scores each candidate spot by the coverage it would give when the drone *arrives* there and for the following minute, using only what could be observed (positions now, movement in the last second and, in announced designs, the operator's message). Tests show it never reads the future.

Table 5.9 Room confirmed on fresh scenarios (seeds 1000-1019, 20 per design; first number: bar of B6, B7, B8; second: bar also including B6fc with the announcement; pass 0.02)

Sites apart	Sudden switch	Over 1 minute	Over 3 minutes
1.0 km	0.016 / 0.013 (no)	0.005 / 0.005 (no)	0.004 / 0.004 (no)
1.5 km	**0.056 / 0.038 (yes)**	**0.029 / 0.029 (yes)**	0.012 / 0.012 (no)
2.0 km	**0.091 / 0.046 (yes)**	**0.071 / 0.047 (yes)**	**0.029 / 0.029 (yes)**

Figure 5.13: Room for a pilot that thinks ahead, on fresh scenarios, against both bars

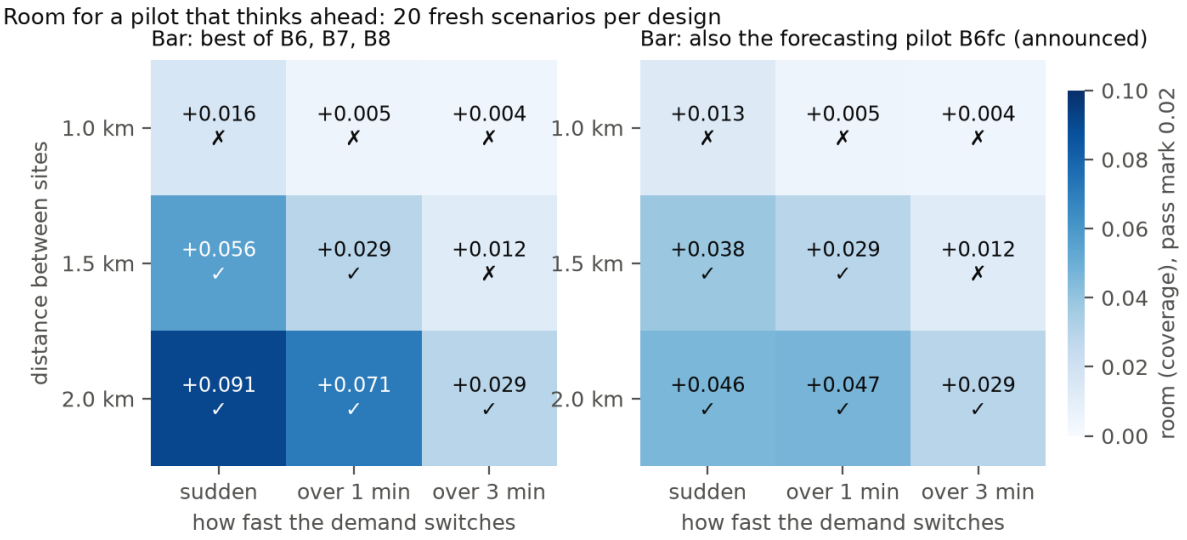


Figure: room_fresh_seeds.png

5 of 9 designs are confirmed; two that passed on the design scenarios (1.0 km sudden, 1.5 km over 3 minutes) did not, which is exactly why fresh scenarios were required. With a 13 Wh battery only the two 2 km designs (sudden, 1 minute) keep room. With the announcement, B6fc captures 36-52% of the room in the sudden and 1-minute designs, but in slow 3-minute switches it does worse than plain B6 because it leaves too early. A departure-time version (B6dep: stay until the best moment, then fly) narrowed the 1.5 km designs below 0.02 on 4 scenarios, leaving **the 2 km designs as the clear place where an AI could win (+0.04 to +0.09)**; these became the G5 training cells.

In progress (expected 24-25 Oct 2026): the anticipation rule. *Not a result yet.* What is being done: one number built from the distance between sites, the drone's speed, the switch duration and the mission length, intended to predict when anticipation (and so any AI) can beat greedy re-planning, together with the departure-time pilot and a three-part breakdown of any AI's shortfall (foresight, information, learning). How it will be judged (pre-registered, D-56): the rule is fitted on the 9 designs above, and its predictions for 8 new designs are written down before those designs are run. What would confirm it: the predicted "room / no room" verdicts match the measured clairvoyant room on the new designs. What would refute it: the predictions miss on the new designs. A quick search found no published equivalent; this still has to be checked in the literature review.

5.8 RL on competing sites: Gate G5, round 1 (negative result)

Round 1 (pre-registered, D-55) trained E0's design plus the event announcement, with fly-to-spot (`goto`) or move-by (`moveby`: "move this far", 0 = stay, built to stop the "flies past the spot" fault) steering, two training seeds each, 150,000 decisions per run (about 3.5 h each, on Kaggle), on the two 2 km announced cells. The bar is, per scenario, the best of B6, B7, B8 and B6fc. All runs use adaptive NOMA (eps 0.05, rescue first) and the acceleration limit; each agent was also evaluated with optimised OMA.

*Table 5.10 G5 round 1 (validation, 20 scenarios; * = CI excludes 0)*

Cell (bar)	Agent	Coverage (NOMA)	Agent minus bar	Coverage with OMA	NOMA minus OMA
sudden switch (0.756)	goto, seed 0	0.673	-0.083*	0.556	+0.117*
sudden switch	goto, seed 1	**0.680**	-0.076*	0.547	+0.133*
sudden switch	moveby, seed 0	0.671	-0.085*	0.588	+0.083*
sudden switch	moveby, seed 1	0.669	-0.087*	0.585	+0.084*
switch over 60 s (0.753)	goto, seed 0	0.673	-0.080*	0.555	+0.118*
switch over 60 s	goto, seed 1	**0.680**	-0.073*	0.552	+0.129*
switch over 60 s	moveby, seed 0	0.616	-0.137*	0.522	+0.094*
switch over 60 s	moveby, seed 1	0.629	-0.124*	0.528	+0.101*

Every agent is significantly below the bar (best 0.680 against 0.756 and 0.753), and also below plain B6 (by 0.03 to 0.10). The forecasting pilot B6fc (0.748 in both cells) carries almost all of the bar. The `moveby` steering did not help (equal on the sudden design, worse on the 1-minute design). **NOMA stays visible here too:** the same agents serve 0.08 to 0.13 more people with adaptive NOMA than with tuned OMA (all significant; the agents were trained with NOMA, so their OMA runs are outside what they learned).

What it means. The room exists, but an AI that has to learn *everything* (where to fly as well as when) from rough maps does not reach it with this budget; it does not even react as well as the search pilot. The room is mostly about **one decision: when to leave for the other site**. Round 2 of the full AI (5 more seeds) was therefore dropped as uninformative (D-56).

In progress (expected 24-25 Oct 2026): the hybrid timing AI. *Not a result yet.* What is being done: a hybrid in which the search pilot B6 decides *where* (position and altitude) and the AI learns only *when* to leave for the announced site, the one decision where the room is; it is disclosed as a hybrid (D-56). How it will be judged: on the same two 2 km cells, against the same pre-registered bar (best of B6, B7, B8, B6fc per scenario, D-55), with paired confidence intervals on the 20 validation scenarios and NOMA vs OMA reported for the agent. What would confirm H2 in this regime: the hybrid beats the bar significantly in at least one cell, and does not beat B6 on a no-room cell (negative control). What would refute it: it stays at or below the bar, as the full AI did.

5.9 Sim-to-sim: does the simplified flight model change the results?

The same pilots flew the same validation scenarios (seeds 100-104) in the point-mass game and with the simulated quadcopter and autopilot (Section 3.10), in calm air and in 5 m/s wind with gusts.

Table 5.11 Sim-to-sim check (5 scenarios per cell)

Cell	Game coverage	Quadcopter, calm	Quadcopter, wind	Coverage gap (calm / wind)
B6 on `hard`	0.694	0.691	0.691	-0.003 / -0.003
B6 on `sites_d20_t0_ann`	0.632	0.626	0.626	-0.006 / -0.006
AI E0 on `rl_static`	0.447	0.436	0.435	-0.010 / -0.012

Flying with a quadcopter model, autopilot lag and a 30 degree tilt limit costs only 0.003 to 0.012 coverage, and wind changes the result by at most 0.002 (the autopilot holds its spot within about a metre). Pilots built for the point game would overshoot their target on a realistic drone unless they brake in time; with the acceleration limit they do (median 1.8 m off target). So **the coverage conclusions carry over to a flying vehicle model** (a simulation-to-simulation check, not a real flight). **The quadcopter's energy numbers are not trustworthy yet:** the Zeng formula makes wobbling and wind look cheaper than hovering, so battery results on the quadcopter need more work. Only 5 scenarios per cell, and not validated against a real drone or ArduPilot/PX4 software-in-the-loop.

Figure 5.14: The 3D flight page: B6 flown by the simulated quadcopter in wind on a 2 km competing-sites scenario (validation seed 100), 170 s into the mission, crossing towards the newly active site (grey = people not counted now, at the other site); one scenario, for illustration only

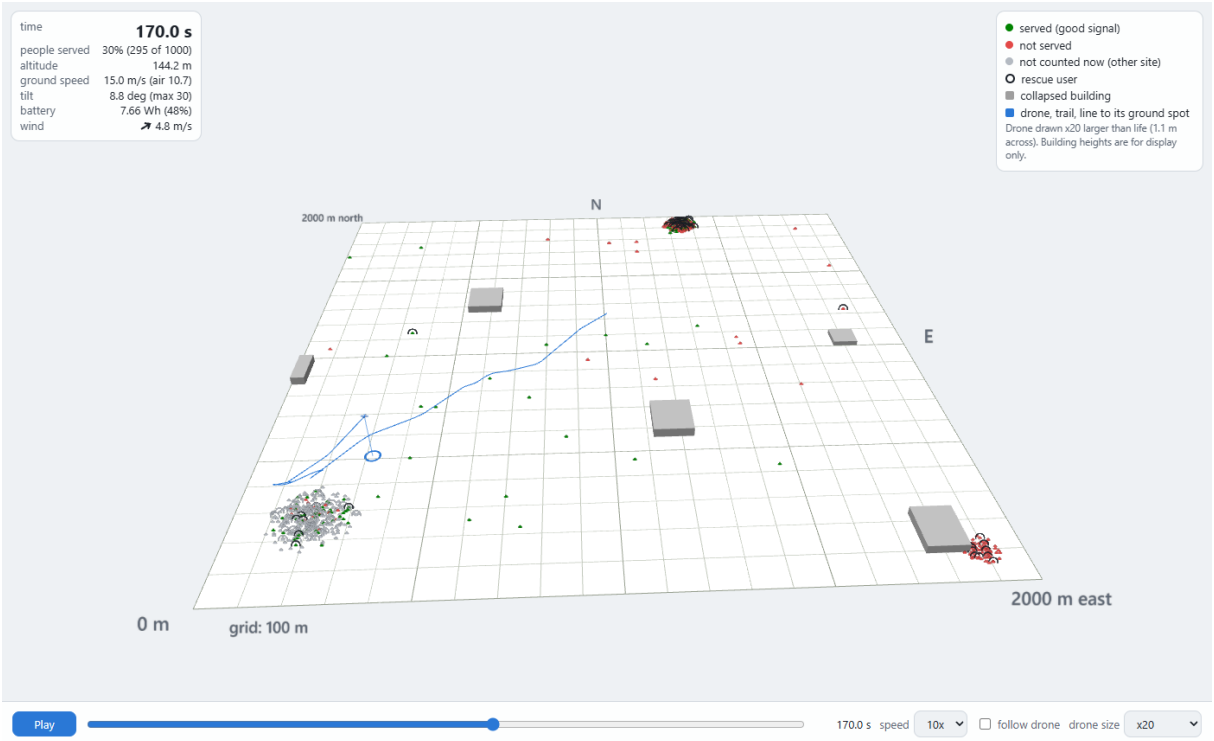


Figure: fig_3d_flight.png

5.10 Value of location information

In progress (expected 24-25 Oct 2026). *Not a result yet.* After a disaster the drone *is* the network, so phones that attach to it can report their position. What is being done: measuring the share, accuracy and age of phone position reports a planner needs to reach 98% of the coverage it gets with full information (D-56). How it will be judged: the search pilots are run with degraded location information (fewer phones reporting, noisier positions, older reports) on the same scenarios and compared, paired, with the full-information run; the still-crowd information ladder (Section 5.5) is the first data point. What would make it a useful design rule: a clear threshold (for example "x% of phones, y m accuracy, z s old is enough"); what would weaken it: coverage that falls smoothly with no usable threshold, or a threshold that depends strongly on the scenario.

5.11 Real-place scenario

Planned, not started; date not fixed. A scenario based on a real area (buildings and shelters from OpenStreetMap, with the required credit line), labelled "a scenario based on ...", with crowds reconstructed and never presented as real survivor data. By the owner's decision it is a demonstration and a final test of the anticipation rule, not a headline result (D-56).

Chapter 6. Discussion

6.1 Where learning helps and where it does not

The results support a split of the control problem by time scale, which a communications engineer would make anyway.

Table 6.1 Where each tool fits

Layer	Time scale	Best tool in our results
Radio resource allocation (pairing, power per pair)	every second	Closed form and optimisation: adaptive NOMA (Section 5.3)
Placement for the users as they are now	every 10 s	Search: B6 (grid + altitude) is near-optimal for walking crowds (Section 5.7)
Battery strategy (hover or circle)	minutes	A one-line rule: B8 (Section 5.4)
Mission timing when demand jumps between distant sites	minutes	The only place with confirmed room (2 km designs, +0.04 to +0.09); the full AI has not captured it; a hybrid timing AI is in progress (Section 5.8)

"Do not learn what you can compute." On still crowds, a search with the AI's own inputs reaches B2's level, so a learner could in principle pass G4, but end-to-end RL did not turn the maps into a precise spot. On walking crowds no controller can gain more than 0.002 over greedy re-planning. Where room exists (competing sites, 2 km), a forecasting pilot already captures most of it and the first learning agents did not. This is the "where it does not help" half of H2, which the hypothesis explicitly asks us to report.

6.2 H2 so far

H2 is **not supported** by the evidence so far: the AI did not match B2 on still crowds (G4), and in G5 round 1 every agent was significantly below the bar. The clairvoyant method explains part of this (no room in walking-crowd scenarios) and located the regime where room exists. Whether a hybrid agent that only decides when to leave closes the gap there is being tested now (Section 5.8, in progress); until then the honest framing is that **physics-based search is the main controller and learning helps, if at all, only in specific regimes**.

6.3 Limitations

- **Model limits:** no building heights or terrain; obstacles are 2D rectangles with a fixed 15 dB loss; no backhaul; no interference from other cells; no bit-error rate; the turning-energy term is our own extension; walking speeds, surge behaviour and minimum rates are engineering assumptions; fading is off in the main runs.
- **Information:** the baselines know every person's true position; in reality survivors' positions are unknown and must be estimated from phone signals. The AI's maps assume a density estimate that a real drone would also have to build.
- **NOMA practicality:** ordinary phones are believed not to support NOMA decoding (to be checked); 10-17% of adaptive NOMA pairs give the weak user less than half the power, which practical receivers may not accept (no clamp tested).
- **Regulation:** our altitude range (50-300 m) is above the 120 m limit that drone rules commonly set for routine flights [PENDING/VERIFY: cite India's Drone Rules 2021 and the DGCA airspace map]; results at the lower altitudes need a re-run.
- **Statistics:** several experiments use 12-20 scenarios per cell; the sim-to-sim check uses 5.
- **Quadcopter energy:** the coverage results carry over to the quadcopter model, but its battery numbers are not yet trustworthy (Section 5.9).
- **Single drone:** H4 was not attempted.

6.4 Threats to validity

- **Internal:** a bug could favour one pilot. Mitigations: one shared `env.step()`, paired comparisons on identical scenarios, tests that new features leave old results bit-exact, a negative control (an AI must not beat B6 where the clairvoyant shows no room), and independent reviews that found real problems (an unfair OMA reference, the shaking exploit, the forecasting pilot leaving too early).
- **Construct:** coverage counts users at their minimum rate; it ignores how much more than the minimum they get (we report total rate and fairness alongside). The minimum rates are not signed off.
- **External:** the data are synthetic; absolute coverage numbers do not transfer to a real disaster, only the comparisons and qualitative lessons do. The Al-Hourani parameters are from secondary sources; MATLAB parity (Gate G1) is not done.
- **Conclusion:** pass marks were written before results (G4, competing sites, G5) and the test seeds are untouched, so the verdicts cannot have been tuned to the outcome.

Chapter 7. Conclusion and future work

7.1 Conclusion

We built an open, tested simulation-based planning tool for a UAV base station serving disaster survivors with a NOMA downlink, and used it to test four hypotheses fairly.

Table 7.1 Hypotheses: verdicts

Hypothesis	Verdict	Evidence
H1 adaptive beats fixed	**Supported** where area is large and power scarce	`hard`: 0.741 vs 0.428 (Table 5.1); small gap in default scenarios (0.914 vs 0.882)
H2 RL beats heuristics only under dynamics and battery limits	**Not supported so far** (hybrid timing AI in progress)	G4 not met (0.576 < 0.580); G5 round 1 all agents below the bar (best 0.680 vs 0.756); no room on walking crowds (+0.002); room confirmed only on 2 km competing sites
H3 NOMA beats OMA	**Supported**, with an honest trade-off	Adaptive NOMA +0.03 to +0.05 over equally optimised OMA for every pilot; fixed NOMA loses coverage at 30 dBm (232 kbps ceiling)
H4 multi-UAV	**Not attempted**	Stretch goal

Further findings: most of a search pilot's lead came from choosing the altitude; circling at the minimum-power speed serves about 26% more people per battery; the clairvoyant headroom test tells in advance whether a scenario leaves any room for a learning controller, and room for it was confirmed on fresh scenarios only where demand jumps between sites 2 km apart; and a quadcopter with an autopilot flies our plans within 0.012 coverage of the simplified game.

7.2 Future work

1. **Finish what is in progress (expected 24-25 Oct 2026):** the anticipation rule (fitted on 9 designs, tested on 8 new ones with predictions written first), the hybrid timing AI against the pre-registered bar, and the value of location information; then the final evaluation (negative control, one read of the test seeds) and this report.
2. **Connect-and-locate survivors from attached phones.** A drone base station knows which phones are connected and can estimate where they are from signal strength or timing. Replacing the "true positions" of the baselines and the density maps of the AI with such estimates closes the largest realism gap; the value-of-location-information study (Section 5.10) is the first step, and it is also a possible product (business track).
3. **Real places:** OpenStreetMap buildings and shelters as obstacles and targets, population density for crowd placement, flood extents and elevation; labelled "a scenario based on ...", never real survivor data, since survivor positions are not published (planned as a demonstration, Section 5.11).
4. **Tethered drones:** a tether supplies power and backhaul, removing the battery limit and the backhaul gap, at the cost of a fixed ground station and a limited radius; drone cell sites of this kind (for example AT&T's tethered "Flying COW") have been reported in news sources after hurricanes [VERIFY citation].
5. **Legal altitude limits:** re-run the main comparisons with the ceiling at 120 m above ground and the corresponding permission rules [VERIFY: Drone Rules 2021].
6. **Disaster events and realism:** rising flood, evacuation, aftershocks, drone failure; wind, GPS error and command delay; a safety layer (geofence, battery reserve, return home, fallback to B6).
7. **NOMA as a decision of the AI**, need-aware pairing, bit-error rate, and a bench test with software-defined radios.
8. **Validation:** read the Al-Hourani paper, MATLAB parity, sign-off of the minimum rates and their sensitivity runs, and one real measurement of the signal against height.

9. **Multi-UAV control (H4)** and a flight in ArduPilot SITL over MAVLink.
10. **Business track:** the most realistic product for a student team is software only, a planning and rehearsal tool for emergency teams or telecom operators; the next step is to interview 5-10 practitioners before building more.

References

[PENDING: every reference below must be checked against the original by a team member (see literature.md, all marked VERIFY). Journal volume and page details marked "verify" were recalled, not confirmed online.]

[1] A. Al-Hourani, S. Kandeepan, and S. Lardner, "Optimal LAP altitude for maximum coverage," *IEEE Wireless Commun. Lett.*, vol. 3, no. 6, pp. 569-572, Dec. 2014, doi: 10.1109/LWC.2014.2342736.

[2] Y. Zeng, J. Xu, and R. Zhang, "Energy minimization for wireless communication with rotary-wing UAV," *IEEE Trans. Wireless Commun.*, vol. 18, no. 4, pp. 2329-2345, 2019 (verify); arXiv:1804.02238.

[3] Y. Zeng and R. Zhang, "Energy-efficient UAV communication with trajectory optimization," *IEEE Trans. Wireless Commun.*, vol. 16, no. 6, pp. 3747-3760, 2017 (verify); arXiv:1608.01828.

[4] M. Mozaffari, W. Saad, M. Bennis, Y.-H. Nam, and M. Debbah, "A tutorial on UAVs for wireless networks: Applications, challenges, and open problems," *IEEE Commun. Surveys Tuts.*, 2019; arXiv:1803.00680.

[5] R. I. Bor-Yaliniz, A. El-Keyi, and H. Yanikomeroglu, "Efficient 3-D placement of an aerial base station in next generation cellular networks," in *Proc. IEEE ICC*, 2016 (venue verify); arXiv:1603.00300.

[6] M. Alzenad, A. El-Keyi, F. Lagum, and H. Yanikomeroglu, "3-D placement of an unmanned aerial vehicle base station (UAV-BS) for energy-efficient maximal coverage," *IEEE Wireless Commun. Lett.*, 2017 (venue verify); arXiv:1705.03415.

[7] J. Lyu, Y. Zeng, R. Zhang, and T. J. Lim, "Placement optimization of UAV-mounted mobile base stations," *IEEE Commun. Lett.*, vol. 21, no. 3, pp. 604-607, 2017; arXiv:1612.01047.

[8] Y. Saito, Y. Kishiyama, A. Benjebbour, T. Nakamura, A. Li, and K. Higuchi, "Non-orthogonal multiple access (NOMA) for cellular future radio access," in *Proc. IEEE VTC Spring*, 2013, doi: 10.1109/VTCSpring.2013.6692652.

[9] Z. Ding, P. Fan, and H. V. Poor, "Impact of user pairing on 5G nonorthogonal multiple-access downlink transmissions," *IEEE Trans. Veh. Technol.*, vol. 65, no. 8, pp. 6010-6023, Aug. 2016; arXiv:1412.2799.

[10] Y. Liu, Z. Qin, Y. Cai, Y. Gao, G. Y. Li, and A. Nallanathan, "UAV communications based on non-orthogonal multiple access," *IEEE Wireless Commun.*, 2019; arXiv:1809.05767.

[11] F. Cui, Y. Cai, Z. Qin, M. Zhao, and G. Y. Li, "Multiple access for mobile-UAV enabled networks: Joint trajectory design and resource allocation," *IEEE Trans. Commun.*, 2019.

[12] X. Chen, Z. Yang, N. Zhao, Y. Chen, J. Wang, Z. Ding, and F. R. Yu, "Secure transmission via power allocation in NOMA-UAV networks with circular trajectory," *IEEE Trans. Veh. Technol.*, vol. 69, no. 9, pp. 10033-10045, Sep. 2020, doi: 10.1109/TVT.2020.3005120.

[13] R. Zhong, X. Liu, Y. Liu, and Y. Chen, "NOMA in UAV-aided cellular offloading: A machine learning approach," arXiv:2011.14776, 2020.

[14] A. Benfaid, N. Adem, and B. Khalfi, "AdaptSky: A DRL based resource allocation framework in NOMA-UAV networks," arXiv:2107.01004, 2021.

[15] X. Liu, Y. Liu, Y. Chen, and L. Hanzo, "Trajectory design and power control for multi-UAV assisted wireless networks: A machine learning approach," arXiv:1812.07665, 2018 (journal version verify).

[16] R. Ghanavi, E. Kalantari, M. Sabbaghian, H. Yanikomeroglu, and A. Yongacoglu, "Efficient 3D aerial base station placement considering users mobility by reinforcement learning," in *Proc. IEEE WCNC*, 2018; arXiv:1801.07472.

[17] Y. Hu, M. Chen, W. Saad, H. V. Poor, and S. Cui, "Meta-reinforcement learning for trajectory design in wireless UAV networks," arXiv:2005.12394, 2020.

[18] D. Rizvi and D. Boyle, "Multi-agent reinforcement learning with action masking for UAV-enabled mobile communications," arXiv:2303.16737, 2023.

[19] R. Wang, E. M. Ghoura, O. Alhussein, Y. Yang, J. Ren, S. Xu, and S. Muhaidat, "Two-layer reinforcement learning-assisted joint beamforming and trajectory optimization for multi-UAV downlink communications," arXiv:2601.12659, 2026.

[20] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," arXiv:1707.06347, 2017.

[21] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine, "Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor," in *Proc. ICML*, 2018; arXiv:1801.01290.

[22] L. Pinto, M. Andrychowicz, P. Welinder, W. Zaremba, and P. Abbeel, "Asymmetric actor critic for image-based robot learning," arXiv:1710.06542, 2017.

[23] A. Raffin, A. Hill, A. Gleave, A. Kanervisto, M. Ernestus, and N. Dormann, "Stable-Baselines3: Reliable reinforcement learning implementations," *J. Mach. Learn. Res.*, vol. 22, no. 268, pp. 1-8, 2021.

[24] M. Towers *et al.*, "Gymnasium: A standard interface for reinforcement learning environments," arXiv:2407.17032, 2024.

[25] P. Henderson, R. Islam, P. Bachman, J. Pineau, D. Precup, and D. Meger, "Deep reinforcement learning that matters," in *Proc. AAAI*, 2018; arXiv:1709.06560.

[26] R. Agarwal, M. Schwarzer, P. S. Castro, A. Courville, and M. G. Bellemare, "Deep reinforcement learning at the edge of the statistical precipice," in *Proc. NeurIPS*, 2021; arXiv:2108.13264.

[27] S. A. W. Shah, T. Khattab, M. Z. Shakir, M. G. Khafagy, and M. O. Hasna, "Small cell association with networked flying platforms: Novel algorithms and performance bounds," arXiv:1802.01117, 2018.

[28] S. A. W. Shah, T. Khattab, M. Z. Shakir, and M. O. Hasna, "A distributed approach for networked flying platform association with small cells in 5G+ networks," arXiv:1705.03304, 2017.

[29] S. Holm, "A simple sequentially rejective multiple test procedure," *Scand. J. Statist.*, vol. 6, no. 2, pp. 65-70, 1979 (verify).

[30] R. Jain, D. Chiu, and W. Hawe, "A quantitative measure of fairness and discrimination for resource allocation in shared computer systems," DEC Research Report TR-301, 1984 (verify).

[31] J. Kennedy and R. Eberhart, "Particle swarm optimization," in *Proc. IEEE Int. Conf. Neural Networks (ICNN)*, 1995 (verify).

[32] The UAV-NOMA Project Book, `docs/book/` in the project repository (GitHub `vathsa1703/Major_project`), 2026.